



دانشگاه تربیت مدرس

دانشکده علوم ریاضی

پایان نامه دوره کارشناسی ارشد علوم کامپیوتر

روش های عمیق مبتنی بر مبدل های بینایی در
تحلیل داده های تصویری

توسط

سید محمد بادزهره

استاد راهنما

آقای دکتر منصور رزقی

تابستان ۱۴۰۴

تقدیم به

پدر بزرگوار و مادر مهربانم و برادر عزیزم
آن‌ها که از خواسته‌هایشان گذشتند، سختی‌ها را به جان خریدند و خود را سپر بلای مشکلات و
ناملایمات کردند تا من به جایگاهی که اکنون در آن ایستاده‌ام برسم.

قدردانی

از استاد کرامت‌دور، جناب آقای دکتر رزقی که بارها بنامی‌های دلسوزانه و ارزشمند خود، همواره در مسیر تحقیق این پایان‌نامه یار و راهنمای من بودند، نهایت سپاس و قدردانی را دارم.

از خانواده عزیزم که با محبت بی‌پایان، صبوری و حمایت‌های بی‌دریغ‌شان، همواره پشتیبان من در طی این مسیر سخت و پرچالش بودند، صمیمانه سپاسگزارم.

چکیده

مبدل‌های بینایی در سال‌های اخیر به یکی از ساختارهای اصلی در مدل‌های یادگیری عمیق برای پردازش تصاویر تبدیل شده‌اند. با این حال، ساختارهای کلاسیک این مبدل‌ها معمولاً شامل لایه‌های کاملاً متصل هستند که نیازمند تبدیل تانسورهای چندبعدی ورودی به بردارهای مسطح می‌باشند. این فرآیند منجر به افزایش چشم‌گیر پارامترها و از بین رفتن ساختار فضایی و روابط میان‌بعدی داده می‌شود. در این پژوهش، چارچوبی تانسوری برای طراحی مدل مبدل تصویری پنجره متحرک^۱ پیشنهاد شده است که با بهره‌گیری از لایه‌های فشرده‌سازی تانسوری^۲ و رگرسیون تانسوری^۳، نگاشت‌های چندخطی میان ابعاد داده را مدل‌سازی می‌کند. این روش با حفظ ساختار چندبعدی تصویر، موجب کاهش قابل توجه تعداد پارامترها، حفظ اطلاعات ساختاری، و افزایش تفسیرپذیری مدل می‌شود. نتایج حاصل از پیاده‌سازی و ارزیابی مدل در مجموعه داده‌های استاندارد نشان می‌دهد که استفاده از ساختارهای تانسوری نه تنها پیچیدگی محاسباتی را کاهش می‌دهد، بلکه دقت طبقه‌بندی را نیز بهبود می‌بخشد.

^۱swin transformer

^۲TCL

^۳TRL

فهرست مطالب

و	فهرست جداول
ز	فهرست تصاویر
۱	پیش‌گفتار
۲	۱ مقدمه
۲	۱.۱ مروری بر تاریخچه هوش مصنوعی
۲	۱.۱.۱ آغاز هوش مصنوعی
۳	۲.۱.۱ دوره طلایی و پیشرفت‌های اولیه
۳	۳.۱.۱ ظهور عصر تاریک هوش مصنوعی
۳	۴.۱.۱ بازگشت هوش مصنوعی و تحولات جدید
۴	۲.۱ انواع مدل یادگیری ماشین و شبکه‌های عصبی
۴	۱.۲.۱ یادگیری ماشین: مروری کلی
۴	۲.۲.۱ تقسیم‌بندی‌های اصلی در یادگیری ماشین
۵	۳.۲.۱ یادگیری نظارت‌شده
۷	۴.۲.۱ یادگیری بدون نظارت
۷	۵.۲.۱ یادگیری تقویتی

۶.۲.۱	معرفی چند مدل از الگوریتم یادگیری کلاسیک	۸
۷.۲.۱	نزدیک‌ترین همسایه	۹
۸.۲.۱	ماشین بردار پشتیبان	۱۰
۹.۲.۱	بیز ساده	۱۱
۱۰.۲.۱	شبکه‌های عصبی پیش‌خور	۱۲
۱۱.۲.۱	شبکه‌های عصبی پیچشی	۱۳
۱۲.۲.۱	شبکه‌های عصبی بازگشتی و شبکه‌های حافظه بلندمدت کوتاه‌مدت	۱۵
۱۳.۲.۱	شبکه‌های عصبی بازگشتی	۱۵

۲ پیشینه پژوهش

۱۶	پیشینه پژوهش	
۱۶	۱.۲ مشکلات ترجمه ماشینی و مبدل‌ها	
۱۷	۲.۲ ظهور ترانسفورمرها	
۱۷	۳.۲ معماری مبدل‌ها	
۱۹	۱.۳.۲ جاسازی	
۲۰	۲.۳.۲ جاسازی موقعیتی	
۲۱	۳.۳.۲ توجه	
۲۴	۴.۳.۲ توجه چندسر	
۲۶	۵.۳.۲ اتصال باقیمانده	
۲۷	۴.۲ نرمال‌سازی لایه‌ها	
۲۹	۱.۴.۲ اتصال باقیمانده	
۳۰	۵.۲ رمزگشا	
۳۱	۶.۲ توجه چندسری ماسک‌شده	
۳۲	۷.۲ مثال عددی توجه ماسک‌شده	

۳۳	۸.۲	مبدل‌های بینایی
۳۳	۱.۸.۲	جاسازی پچ‌ها در مبدل‌های بینایی
۳۴	۲.۸.۲	شکل پچ‌ها:
۳۴	۳.۸.۲	تعداد پچ‌ها:
۳۵	۴.۸.۲	بردارکردن هر پچ
۳۵	۹.۲	اعمال لایه خطی
۳۷	۱.۹.۲	توکن کلاس‌بندی
۳۸	۲.۹.۲	رمزگذار در مبدل‌های بینایی
۳۹	۱۰.۲	مبدل پنجره‌ای متحرک
۴۰	۱.۱۰.۲	قطعه‌بندی پچ در مبدل پنجره متحرک
۴۲	۲.۱۰.۲	توجه چندسر پنجره‌ای
۴۲	۳.۱۰.۲	توجه
۴۴	۴.۱۰.۲	پنجره متحرک جابه‌جاشده
۴۶	۵.۱۰.۲	پرسپترون چندلایه
۴۷	۶.۱۰.۲	ترکیب پچ‌ها

۵۴	۱.۰.۳	تجزیه تاکر
۵۵	۲.۰.۳	لایه فشرده‌سازی تانسوری
۵۷	۳.۰.۳	لایه رگرسیون تانسوری (TRL)
۵۸	۴.۰.۳	چرا در مبدل‌های بینایی از تانسور استفاده می‌کنیم؟
۵۹	۵.۰.۳	روش تانسوری مبدل پنجره متحرک:
۵۹	۶.۰.۳	پیاده‌سازی مرحله تعبیه پچ با استفاده از فشرده‌سازی تانسوری

۶۲	ماژول خودتوجهی چندسری مبتنی بر پنجره به صورت تانسوری	۷.۰.۳
۶۶	توجه علامت دار	۸.۰.۳
۶۸	توجه مبتنی بر پنجره‌های جابه‌جاشده به صورت تانسوری	۹.۰.۳
۶۹	ادغام پیچ‌ها به صورت تانسوری	۱۰.۰.۳
۷۲	مرحله‌ی طبقه‌بندی نهایی با استفاده از رگرسیون تانسوری	۱۱.۰.۳
۷۳	طبقه‌بندی با استفاده از لایه رگرسیون تانسوری	۱۲.۰.۳

۴ نتایج و تحلیل‌ها

۷۵	ارزیابی بر روی مجموعه داده CIFAR-10	۱.۴
۷۵	خلاصه نتایج کمی	۱.۱.۴
۷۶	تحلیل نتایج	۲.۱.۴
۷۶	نمایش روند آموزش	۳.۱.۴
۷۷	جمع‌بندی	۴.۱.۴
۷۷	نتایج بر روی دیتاست MNIST	۲.۴
۷۸	خلاصه نتایج کمی	۱.۲.۴
۷۸	نمایش روند آموزش	۲.۲.۴
۷۹	تحلیل و بحث	۳.۲.۴
۷۹	جمع‌بندی	۴.۲.۴
۸۰	نتایج بر روی دیتاست Tiny ImageNet	۳.۴
۸۰	خلاصه نتایج کمی	۱.۳.۴
۸۱	نمایش روند آموزش	۲.۳.۴
۸۱	تحلیل و بحث	۳.۳.۴
۸۲	جمع‌بندی	۴.۳.۴

فهرست مطالب

۵

کتاب‌نامه

۸۴

آ جزئیات مدل‌ها و جدول پارامترها

۹۰

فهرست جداول

۱.۴	مقایسه عملکرد مدل اصلی و مدل پیشنهادی بر روی مجموعه داده CIFAR-10
۷۶	بر حسب دقت‌های Top-1 و Top-5.
۲.۴	مقایسه عملکرد مدل اصلی و مدل تانسوری بر روی MNIST (فقط Top-۱
۷۸	و Top-۵).
۳.۴	مقایسه عملکرد مدل‌ها بر روی Tiny ImageNet.
۸۰	

فهرست تصاویر

۶	کلاس بندی	۱.۱
۶	رگرسیون	۲.۱
۷	خوشه بندی	۳.۱
۸	یادگیری تقویتی	۴.۱
۹	الگوریتم نزدیکترین همسایه	۵.۱
۱۱	الگوریتم ماشین بردار پشتیبان	۶.۱
۱۳	شبکه عصبی پیش خور	۷.۱
۱۴	شبکه عصبی پیچشی	۸.۱
۱۸	معماری ترانسفورمرها	۱.۲
۱۹	جاسازی کلمه‌ای	۲.۲
۲۱	اضافه کردن جاسازی موقعیتی به جاسازی کلمه‌ای	۳.۲
۲۳	نحوه به دست آوردن پرسش، کلید و مقدار	۴.۲
۲۴	توجه	۵.۲
۲۵	توجه چندسر	۶.۲
۳۱	رمزگشا در مبدل‌ها	۷.۲
۳۴	بخش بندی تصاویر	۸.۲

۳۶	۹.۲	مبدل‌های بینایی
۳۸	۱۰.۲	توکن توجه در مبدل‌های بینایی
۴۰	۱۱.۲	مبدل پنجره متحرک
۴۶	۱۲.۲	جابه‌جایی چرخه‌ای
۴۹	۱۳.۲	ادغام پیچ‌ها
۵۲	۱.۳	شبکه عصبی پرسپترون چندلایه
۵۵	۲.۳	لایه فشرده‌ساز تانسوری
۶۱	۳.۳	تعبیه پیچ‌ها
۷۱	۴.۳	ادغام پیچ‌ها
	۱.۴	روند تغییرات دقت Top-1 مدل اصلی Swin-Tiny بر روی مجموعه داده CIFAR-10
۷۷		
	۲.۴	روند تغییرات دقت Top-1 مدل پیشنهادی Tensorized Swin بر روی مجموعه داده CIFAR-10
۷۸		
۷۹	۳.۴	روند دقت Top-1 مدل اصلی Swin-Tiny بر روی MNIST
۸۰	۴.۴	روند دقت Top-1 مدل تانسوری پیشنهادی بر روی MNIST
۸۱	۵.۴	روند دقت Top-1 مدل اصلی Swin-Tiny بر روی Tiny ImageNet
۸۲	۶.۴	روند دقت Top-1 مدل تانسوری با بهینه‌ساز Adam بر روی Tiny ImageNet
۸۳	۷.۴	روند دقت Top-1 مدل تانسوری با بهینه‌ساز AdamW بر روی Tiny ImageNet

پیش‌گفتار

پیش‌گفتار

مبدل‌های بینایی^۴ در سال‌های اخیر توجه زیادی در حوزه بینایی ماشین و یادگیری عمیق به خود جلب کرده‌اند. این مدل‌ها با وجود عملکرد قوی، دارای تعداد زیادی پارامتر هستند و معمولاً ساختار فضایی داده را از بین می‌برند. استفاده از روش‌های تانسوری راه‌حلی برای این مسئله ارائه می‌دهد. در این پژوهش، چارچوبی مبتنی بر ساختارهای تانسوری برای بهینه‌سازی مبدل‌های بینایی طراحی شده است. هدف از این کار، کاهش تعداد پارامترها، حفظ ساختار داده و بهبود تفسیرپذیری مدل است. از لایه‌های فشرده‌سازی تانسوری (TCL) و رگرسیون تانسوری (TRL) برای مدل‌سازی نگاشت‌های چندبعدی استفاده شده است.

این پایان‌نامه شامل چهار فصل است:

- فصل اول: بیان مقدمه، مسئله تحقیق، اهداف پژوهش و اهمیت موضوع. - فصل دوم: مرور پیشینه پژوهش، معرفی ساختار مبدل‌های بینایی و مبدل‌های پنجره متحرک. - فصل سوم: تشریح روش پیشنهادی، طراحی مدل تانسوری و پیاده‌سازی معماری مبدل پنجره متحرک به صورت تانسوری. - فصل چهارم: ارزیابی مدل، تحلیل نتایج حاصل از آزمایش‌ها و مقایسه عملکرد مدل پیشنهادی با مدل‌های پایه.

در پایان، جمع‌بندی، نتیجه‌گیری و پیشنهادهایی برای پژوهش‌های آینده ارائه شده است.

فصل ۱

مقدمه

۱.۱ مروری بر تاریخچه هوش مصنوعی

هوش مصنوعی در سال‌های اخیر با توسعه فناوری‌های مبتنی بر داده و افزایش نیاز به تحلیل هوشمند اطلاعات، دچار جهشی چشم‌گیر شده است. مدل‌های یادگیری عمیق و شبکه‌های عصبی، به‌ویژه در حوزه‌هایی مانند پردازش زبان طبیعی، بینایی ماشین و تحلیل سری‌های زمانی، نقش کلیدی پیدا کرده‌اند. یکی از مهم‌ترین معماری‌های تحول‌آفرین در این مسیر، مدل‌ها^۱ هستند که با تکیه بر مکانیزم توجه، امکان مدل‌سازی وابستگی‌های بلندمدت را فراهم کرده‌اند. درک عمیق این معماری نیازمند آشنایی با روند شکل‌گیری هوش مصنوعی و تکامل الگوریتم‌های یادگیری ماشین است. در این بخش، تاریخچه شکل‌گیری این حوزه به‌طور خلاصه بررسی می‌شود.

۱.۱.۱ آغاز هوش مصنوعی

پیدایش هوش مصنوعی به دهه ۱۹۵۰ بازمی‌گردد؛ زمانی که جان مک‌کارتی^۲ اصطلاح «هوش مصنوعی» را معرفی کرد [۳۹]. هدف اولیه این حوزه، توسعه ماشین‌هایی بود که بتوانند جنبه‌هایی از رفتار انسانی مانند حل مسئله، یادگیری و درک زبان را تقلید کنند. در این دوره، خوش‌بینی

^۱Transformer
^۲John McCarthy

زیادی نسبت به امکان ساخت هوش مصنوعی عمومی^۳ وجود داشت و پروژه‌های متعددی با هدف شبیه‌سازی جامع هوش انسان آغاز شدند [۸، ۴۷].

۲.۱.۱ دوره طلایی و پیشرفت‌های اولیه

دهه‌های ۱۹۵۰ و ۱۹۶۰ را می‌توان دوران طلایی هوش مصنوعی دانست. در این زمان، الگوریتم‌های جستجو، حل مسئله و مدل‌های اولیه پردازش زبان طبیعی توسعه یافتند. همچنین سیستم‌های خبره^۴ و برنامه‌هایی که توانایی انجام بازی‌های فکری مانند شطرنج را داشتند، مورد توجه پژوهشگران قرار گرفتند [۴۶، ۱۴]. این موفقیت‌ها امید دستیابی به هوش مصنوعی عمومی را تقویت می‌کرد.

۳.۱.۱ ظهور عصر تاریک هوش مصنوعی

با وجود پیشرفت‌ها، محدودیت‌های تکنولوژیک دهه ۱۹۷۰—از جمله نبود پردازنده‌های قدرتمند^۵ و کمبود داده مناسب—باعث شد بسیاری از مدل‌ها نتوانند عملکرد مورد انتظار را ارائه دهند. وابستگی شدید الگوریتم‌ها به قوانین دست‌نویس نیز مانع تعمیم‌پذیری آن‌ها می‌شد [۵۲]. این ناکامی‌ها موجب کاهش سرمایه‌گذاری‌ها و رکود گسترده‌ای شد که با عنوان «زمستان هوش مصنوعی»^۶ شناخته می‌شود [۸، ۳۵].

۴.۱.۱ بازگشت هوش مصنوعی و تحولات جدید

در دهه‌های ۱۹۸۰ و ۱۹۹۰، ظهور سیستم‌های خبره و پیشرفت سخت‌افزارها زمینه بازگشت دوباره هوش مصنوعی را فراهم کرد [۱۴]. سیستم‌های خبره با تکیه بر پایگاه‌های قواعد بزرگ، در صنایع مختلف مانند پزشکی، تولید و امور مالی کاربرد پیدا کردند [۴۰]. همزمان، قدرت پردازش کامپیوترها افزایش یافت و هزینه‌های ذخیره‌سازی و محاسبات کاهش پیدا کرد. رشد اینترنت

AGI^۳

Expert Systems^۴

GPU^۵

AI Winter^۶

نیز دسترسی به داده‌های واقعی را بسیار آسان‌تر کرد [۴۷]. این مجموعه عوامل مسیر را برای توسعه الگوریتم‌های یادگیری ماشین و شبکه‌های عصبی مدرن هموار ساخت و پایه‌گذار پیشرفت‌های چشم‌گیر دهه‌های بعدی شد [۵۲].

۲.۱ انواع مدل یادگیری ماشین و شبکه‌های عصبی

یادگیری ماشین و شبکه‌های عصبی به عنوان دو زیرشاخه مهم از هوش مصنوعی، در سال‌های اخیر به‌طور گسترده‌ای مورد توجه پژوهشگران و صنعت قرار گرفته‌اند. این مدل‌ها با هدف یادگیری الگوها و روابط موجود در داده‌ها توسعه یافته‌اند و امروزه در حوزه‌های مختلفی از جمله بینایی ماشین، پردازش زبان طبیعی، پیش‌بینی سری‌های زمانی و داده‌کاوی مورد استفاده قرار می‌گیرند [۴۴، ۴۲، ۴].

۱.۲.۱ یادگیری ماشین: مروری کلی

یادگیری ماشین^۷ شاخه‌ای از هوش مصنوعی است که به مدل‌های محاسباتی این امکان را می‌دهد الگوها را از داده‌ها به شکل خودکار یاد بگیرند و بتوانند تصمیم‌گیری کنند [۴۲، ۱۶]. در واقع، هدف یادگیری ماشین این است که مدل‌ها بتوانند از داده‌ها الگوها و روابط پنهان را استخراج کنند و به نتایج و تصمیم‌های قابل اعتماد دست یابند.

۲.۲.۱ تقسیم‌بندی‌های اصلی در یادگیری ماشین

به طور کلی، یادگیری ماشین به سه دسته اصلی تقسیم می‌شود:

● یادگیری با نظارت^۸

● یادگیری بدون نظارت^۹

Machine Learning^۷

Supervised Learning^۸

Unsupervised Learning^۹

● یادگیری تقویتی^{۱۰}

این طبقه‌بندی در بسیاری از کتاب‌ها و مراجع مهم یادگیری ماشین مطرح شده است [۴، ۴۴].

۳.۲.۱ یادگیری نظارت‌شده

یادگیری نظارت‌شده یکی از رایج‌ترین روش‌ها در یادگیری ماشین شناخته می‌شود که در آن از مجموعه داده‌های برچسب‌گذاری‌شده برای آموزش مدل استفاده می‌کنیم [۲۴]. هدف این الگوریتم تشخیص الگوها در میان داده‌های ورودی است تا بتواند روی داده‌های جدید پیش‌بینی یا طبقه‌بندی انجام دهد. این نوع شامل دو دسته الگوریتم رگرسیون^{۱۱} و کلاس‌بندی^{۱۲} می‌شود.

کلاس‌بندی

در میان روش‌های یادگیری با نظارت، کلاس‌بندی^{۱۳} یکی از پرکاربردترین مسائل محسوب می‌شود. هدف در این مسئله، تخصیص هر نمونه ورودی به یکی از برچسب‌های گسسته از پیش تعریف‌شده است. مدل با استفاده از داده‌های آموزشی که شامل ویژگی‌ها و برچسب صحیح هستند، الگوهای حاکم بر داده را می‌آموزد و تلاش می‌کند بر اساس آن، داده‌های جدید را به دسته مناسب اختصاص دهد. این روش در کاربردهای متنوعی نظیر تشخیص هرزنامه^{۱۴}، شناسایی بیماری‌ها، طبقه‌بندی تصاویر و تحلیل احساسات به‌طور گسترده مورد استفاده قرار می‌گیرد [۴، ۴۴].

رگرسیون

در مقابل، رگرسیون^{۱۵} به مسئله پیش‌بینی مقادیر پیوسته می‌پردازد. در این نوع از یادگیری نظارت‌شده، مدل می‌کوشد رابطه‌ای میان متغیرهای ورودی و خروجی‌های عددی برقرار کند تا بر

Reinforcement Learning^{۱۰}

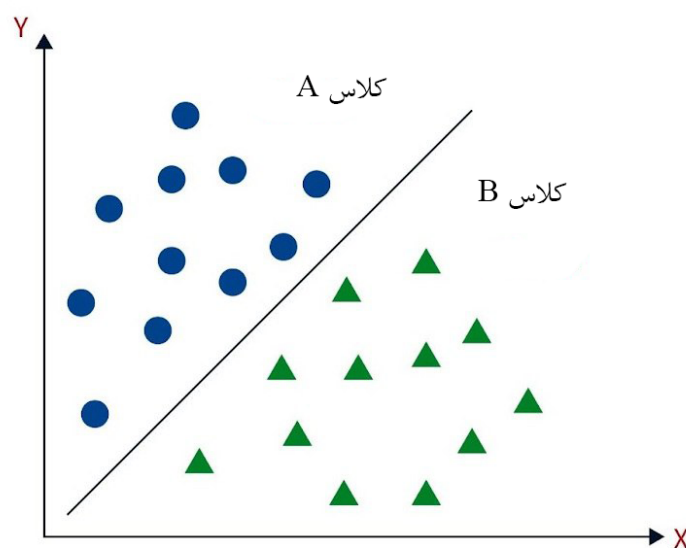
Regression^{۱۱}

Classification^{۱۲}

Classification^{۱۳}

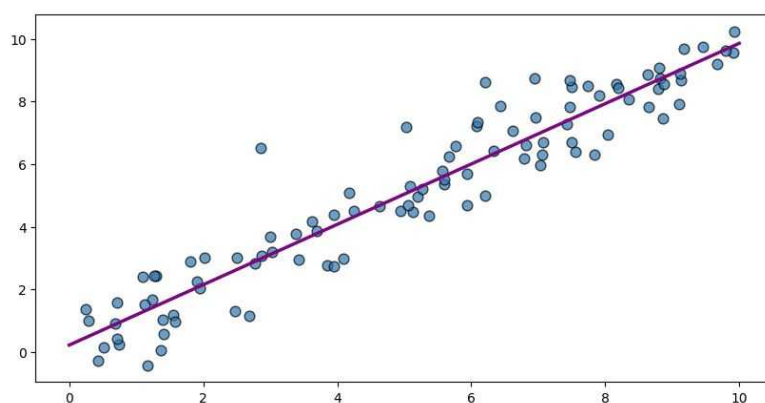
Spam Detection^{۱۴}

Regression^{۱۵}



شکل ۱.۱: کلاس بندی

اساس آن، بتواند خروجی نمونه‌های جدید را تخمین بزند. تفاوت اصلی رگرسیون با کلاس بندی در ماهیت خروجی است؛ به طوری که در رگرسیون، خروجی به جای دسته بندی، یک مقدار عددی خواهد بود. از جمله کاربردهای رایج این نوع مدل ها می توان به پیش بینی قیمت مسکن، تخمین میزان فروش، و پیش بینی وضعیت آب و هوا اشاره کرد [۴۳].

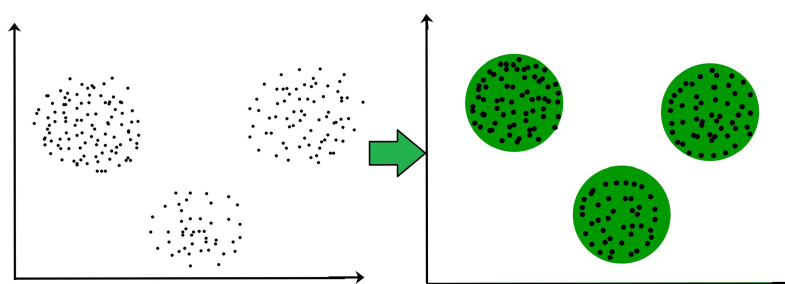


شکل ۲.۱: رگرسیون

۴.۲.۱ یادگیری بدون نظارت

یادگیری بدون نظارت^{۱۶} نوعی از روش‌های یادگیری ماشین است که در آن مدل بدون استفاده از برچسب‌های خروجی، سعی در کشف الگوها، ساختارها یا روابط پنهان میان داده‌ها دارد [۴]. در این رویکرد، داده‌های ورودی تنها شامل ویژگی‌ها هستند و هدف، گروه‌بندی یا استخراج ساختار درونی آن‌ها بدون دانش قبلی از دسته‌بندی صحیح است. برخلاف یادگیری با نظارت که مدل از پاسخ صحیح برای آموزش استفاده می‌کند، در یادگیری بدون نظارت، مدل باید به‌تنهایی ساختارهای معنادار را در داده‌ها کشف کند.

از کاربردهای رایج یادگیری بدون نظارت می‌توان به خوشه‌بندی^{۱۷}، کاهش ابعاد^{۱۸}، آشکارسازی ناهنجاری‌ها^{۱۹}، و استخراج ویژگی‌ها اشاره کرد.



شکل ۳.۱: خوشه بندی

۵.۲.۱ یادگیری تقویتی

یادگیری تقویتی^{۲۰} نوعی یادگیری بر پایه پاداش و تنبیه است که در آن مدل با محیط تعامل می‌کند و بر اساس پاداش یا تنبیه یاد می‌گیرد [۵۵]. برخلاف یادگیری نظارت‌شده و بدون نظارت، یادگیری تقویتی به مدل این امکان را می‌دهد تا از طریق آزمون و خطا بهترین راهکارها را برای انجام یک

^{۱۶} Unsupervised Learning

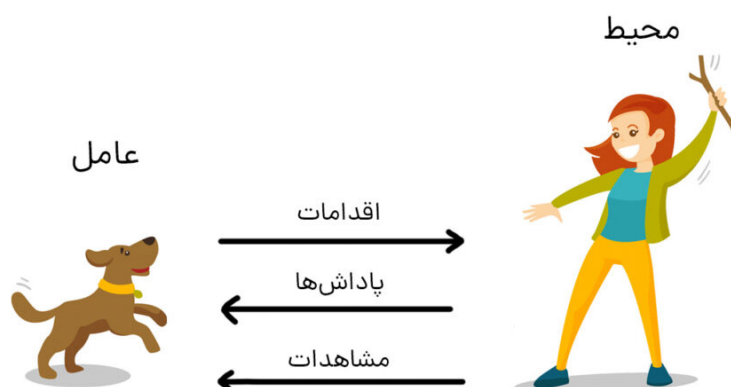
^{۱۷} Clustering

^{۱۸} Dimensionality Reduction

^{۱۹} Anomaly Detection

^{۲۰} reinforcement learning

عمل یاد بگیرد. در این روش، مدل به جای برچسب، از یک تابع پاداش استفاده می‌کند که مشخص می‌کند چه اقداماتی باعث نتیجه بهینه می‌شود. از کاربردهای یادگیری تقویتی می‌توان به بازی‌ها^{۲۱}، کنترل رباتیک^{۲۲} و سیستم‌های توصیه‌گر^{۲۳} اشاره کرد.



شکل ۴.۱: یادگیری تقویتی

۶.۲.۱ معرفی چند مدل از الگوریتم یادگیری کلاسیک

الگوریتم‌های یادگیری کلاسیک، پایه و اساس بسیاری از پیشرفت‌های اولیه در یادگیری ماشین را شکل داده‌اند. این الگوریتم‌ها با وجود سادگی نسبی، در بسیاری از کاربردها همچنان عملکرد قابل قبولی از خود نشان می‌دهند و در بسیاری از سامانه‌های عملیاتی مورد استفاده قرار می‌گیرند.

الگوریتم‌هایی مانند نزدیک‌ترین همسایه^{۲۴}، ماشین بردار پشتیبان^{۲۵}، بیز ساده^{۲۶} و درخت تصمیم^{۲۷} از جمله مشهورترین روش‌های کلاسیک یادگیری هستند که هر یک بر اساس اصول ریاضی و آماری

^{۲۱} Games

^{۲۲} Robotic Control

^{۲۳} Recommender Systems

^{۲۴} k-nearest neighbors algorithm

^{۲۵} Support vector machine

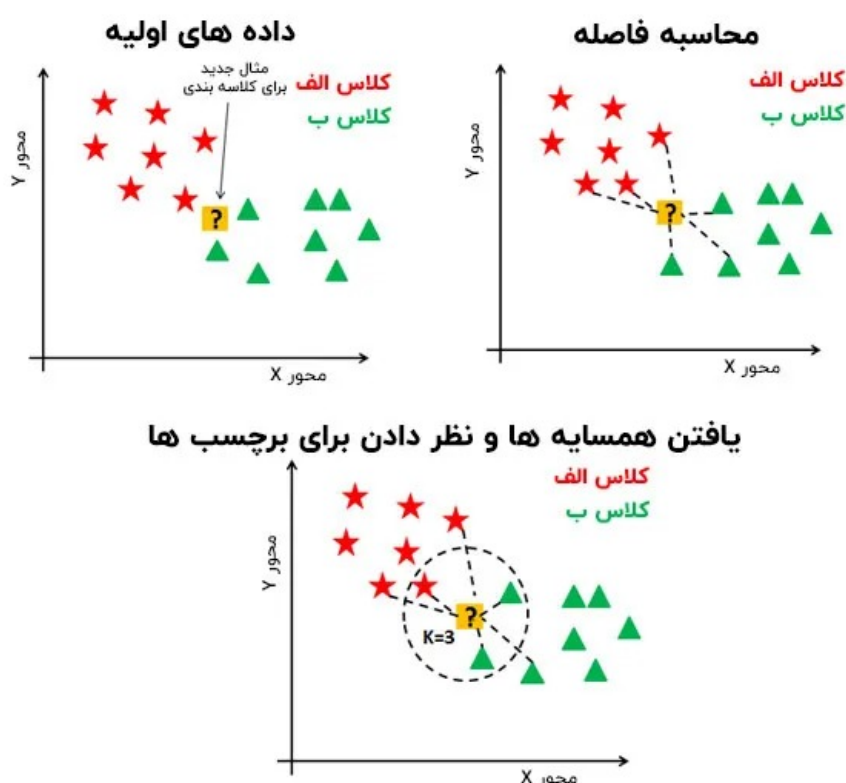
^{۲۶} Naive Bayes

^{۲۷} Decision Tree

متفاوتی طراحی شده‌اند. این مدل‌ها معمولاً برای مسائل طبقه‌بندی یا رگرسیون به کار می‌روند و از مزایایی چون پیاده‌سازی آسان، قابلیت تفسیر بالا و نیاز کمتر به تنظیمات پیچیده برخوردارند. در این بخش، به معرفی اجمالی چند مورد از این الگوریتم‌ها پرداخته می‌شود تا زمینه درک عمیق‌تر روش‌های نوین‌تر یادگیری، مانند شبکه‌های عصبی و مدل‌های عمیق، فراهم گردد.

۷.۲.۱ نزدیک‌ترین همسایه

الگوریتم نزدیک‌ترین همسایه^{۲۸} یکی از روش‌های ساده و درعین حال کارآمد در یادگیری نظارت‌شده است که هم در دسته‌بندی و هم در رگرسیون کاربرد دارد [۷، ۱۲، ۴۲]. این الگوریتم برای پیش‌بینی دسته‌بندی یک نمونه جدید، به k نزدیک‌ترین داده‌ها در فضای ویژگی نگاه می‌کند و بر اساس اکثریت نزدیکی همسایه‌ها، آن را به یک دسته اختصاص می‌دهد.



شکل ۵.۱: الگوریتم نزدیک‌ترین همسایه

^{۲۸}k-Nearest Neighbors

از مهم‌ترین مزایای این الگوریتم می‌توان به سادگی و قابل فهم بودن اشاره کرد؛ چرا که تنها با اندازه‌گیری فاصله بین نقاط داده عمل می‌کند و بدون نیاز به آموزش یک مدل پیچیده، قابل استفاده است [۷]. همچنین، عملکرد خوب در داده‌های با تعداد ویژگی کم از دیگر نقاط قوت آن است، زیرا در مسائلی که تعداد ویژگی‌ها محدود است، این الگوریتم اغلب نتایج مطلوبی ارائه می‌دهد [۲۴].

در مقابل، الگوریتم نزدیک‌ترین همسایه دارای برخی معایب نیز هست. یکی از مهم‌ترین آن‌ها حساسیت به داده‌های پرت است؛ به‌طوری‌که نقاط پرت می‌توانند تأثیر قابل توجهی بر نتایج داشته باشند [۱۲]. علاوه بر این، کندی در داده‌های بزرگ از دیگر مشکلات این روش است، زیرا برای هر نقطه جدید نیاز به محاسبه فاصله با تمام داده‌ها دارد که در مجموعه داده‌های بزرگ منجر به بار محاسباتی بالا می‌شود [۴۲]. در نهایت، این الگوریتم در داده‌های با ابعاد بالا کارایی مناسبی ندارد و با افزایش تعداد ویژگی‌ها عملکرد آن به طور محسوسی افت می‌کند [۴۴].

۸.۲.۱ ماشین بردار پشتیبان

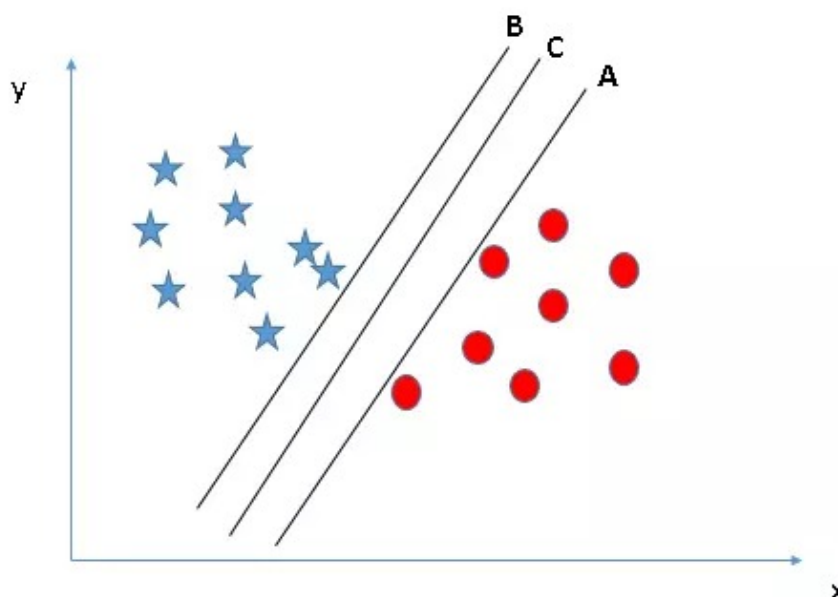
الگوریتم ماشین بردار پشتیبان^{۲۹} با یافتن یک ابرصفحه بهینه، داده‌ها را به کلاس‌های مختلف تقسیم می‌کند [۵۸، ۶]. این الگوریتم یک ابرصفحه به دست می‌آورد که هدف آن حداکثر کردن فاصله میان داده‌های دو کلاس است و به این ترتیب می‌تواند طبقه‌بندی دقیقی داشته باشد.

از مهم‌ترین مزایای ماشین بردار پشتیبان می‌توان به توانایی مقابله با داده‌های پیچیده و ابعاد بالا اشاره کرد؛ این الگوریتم قادر است به خوبی داده‌های چندبعدی و پیچیده را مدیریت کند و در چنین شرایطی همچنان عملکرد دقیقی داشته باشد [۵۸]. علاوه بر این، مقاومت در برابر بیش‌برازش^{۳۰} یکی دیگر از نقاط قوت آن است. با استفاده از توابع هسته^{۳۱}، داده‌های غیرخطی به فضای ویژگی‌های بالاتر نگاشت می‌شوند و این امکان فراهم می‌شود که مرز جداسازی بهتری میان کلاس‌ها ایجاد گردد [۶].

^{۲۹}Support Vector Machine, SVM

^{۳۰}Overfitting

^{۳۱}Kernels



شکل ۶.۱: الگوریتم ماشین بردار پشتیبان

با وجود مزایای ذکر شده، ماشین بردار پشتیبان معایبی نیز دارد. نخستین مسئله، پیچیدگی محاسباتی آن است؛ آموزش این الگوریتم به دلیل نیاز به حل مسائل بهینه‌سازی، در حجم‌های بالای داده بسیار زمان‌بر خواهد بود [۴۴]. همچنین، کارایی پایین در داده‌های پرت از دیگر نقاط ضعف آن است، چرا که وجود تعداد زیادی داده پرت می‌تواند به شکل قابل توجهی دقت مدل را کاهش دهد [۴].

۹.۲.۱ بیز ساده

بیز ساده^{۳۲} مبتنی بر قضیه بیز^{۳۳} است و فرض می‌کند ویژگی‌ها به صورت شرطی مستقل از هم هستند [۴۲، ۱۰]. این مدل برای اولین بار در حوزه پردازش متن به کار رفت و هنوز هم در بسیاری از کاربردها مانند طبقه‌بندی ایمیل و تحلیل احساسات مورد استفاده قرار می‌گیرد [۳۸]. در بیز ساده بر اساس احتمالات محاسبه می‌شود که یک نمونه جدید به کدام دسته تعلق دارد. این الگوریتم بر

^{۳۲} Bayes Naive
^{۳۳} Bayes' theorem

اساس قضیه بیز، احتمال تعلق یک نمونه به هر دسته را به ازای هر ویژگی محاسبه کرده و در نهایت بالاترین احتمال را به عنوان جواب نهایی در نظر می‌گیرد [۴].

از مهم‌ترین مزایای بیز ساده می‌توان به سرعت بالا اشاره کرد؛ چرا که به دلیل محاسبات ساده و فرض استقلال ویژگی‌ها، این الگوریتم بسیار سریع و کم‌حجم بوده و برای مسائل با داده‌های زیاد یا در شرایط نیاز به پاسخ بلادرنگ کارایی بالایی دارد [۳۸]. علاوه بر این، کارایی در داده‌های کوچک از دیگر نقاط قوت آن است، زیرا حتی با وجود حجم محدود داده‌ها، مدل همچنان قادر است عملکرد قابل قبولی ارائه دهد [۴۴].

با وجود مزایا، بیز ساده محدودیت‌هایی هم دارد. یکی از مهم‌ترین آن‌ها فرض استقلال ویژگی‌هاست؛ فرضی که در بسیاری از مسائل واقعی برقرار نیست و می‌تواند باعث کاهش دقت مدل شود [۱۰]. علاوه بر این، این الگوریتم نسبت به کیفیت داده‌ها بسیار حساس است و حساسیت به داده‌های نادرست یا پرت می‌تواند موجب کاهش چشمگیر دقت آن شود [۴].

۱۰.۲.۱ شبکه‌های عصبی پیش‌خور

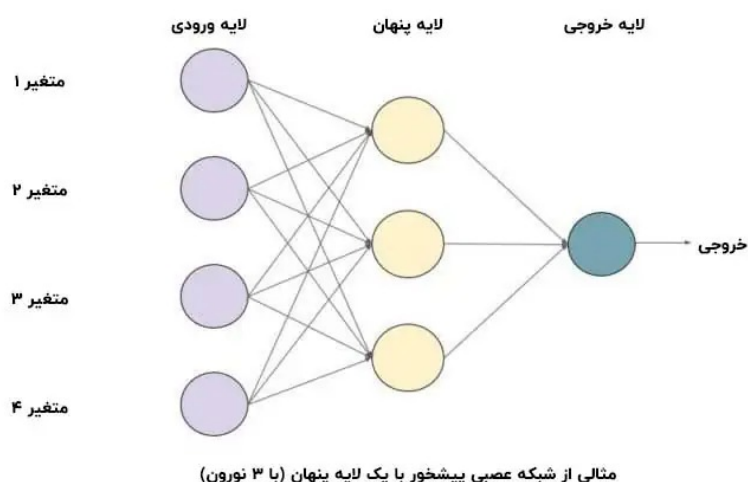
شبکه‌های عصبی پیش‌خور^{۳۴} یا پرسپترون چندلایه^{۳۵} از نخستین مدل‌های یادگیری عمیق هستند که به صورت گسترده در مسائل طبقه‌بندی و رگرسیون به کار گرفته شدند [۴]. این شبکه‌ها از تعدادی لایه ورودی، لایه‌های پنهان و یک لایه خروجی تشکیل می‌شوند. هر نورون در یک لایه با تمامی نورون‌های لایه بعدی ارتباط دارد و با ضرب وزن‌ها، جمع بایاس‌ها و اعمال یک تابع فعال‌سازی، خروجی خود را محاسبه می‌کند. آموزش شبکه از طریق الگوریتم پس‌انتشار خطا^{۳۶} و روش‌های بهینه‌سازی مانند گرادیان کاهشی انجام می‌شود [۵۱].

این شبکه‌ها در مسائل مختلفی مانند تشخیص الگو، پیش‌بینی سری‌های زمانی و تقریب توابع توانسته‌اند عملکرد مناسبی ارائه دهند. با این حال، در داده‌هایی با ابعاد بسیار بالا مانند تصاویر،

^{۳۴} Feedforward Neural Networks

^{۳۵} Multi-Layer Perceptron, MLP

^{۳۶} Backpropagation



شکل ۷.۱: شبکه عصبی پیش خور

این ساختار با مشکلاتی روبه‌رو می‌شود. از یک سو، اتصال کامل میان لایه‌ها باعث انفجار تعداد پارامترها می‌شود؛ برای مثال، یک تصویر رنگی با ابعاد 224×224 پیکسل بیش از ۱۵۰ هزار ویژگی ورودی دارد که اتصال کامل آن به لایه پنهان به میلیون‌ها پارامتر نیاز دارد. از سوی دیگر، شبکه‌های پیش‌خور فاقد سازوکاری برای بهره‌گیری از ساختار مکانی داده‌ها هستند و همه پیکسل‌ها را مستقل از موقعیتشان در تصویر در نظر می‌گیرند.

این محدودیت‌ها زمینه‌ساز ظهور شبکه‌های عصبی پیچشی شدند.

۱۱.۲.۱ شبکه‌های عصبی پیچشی

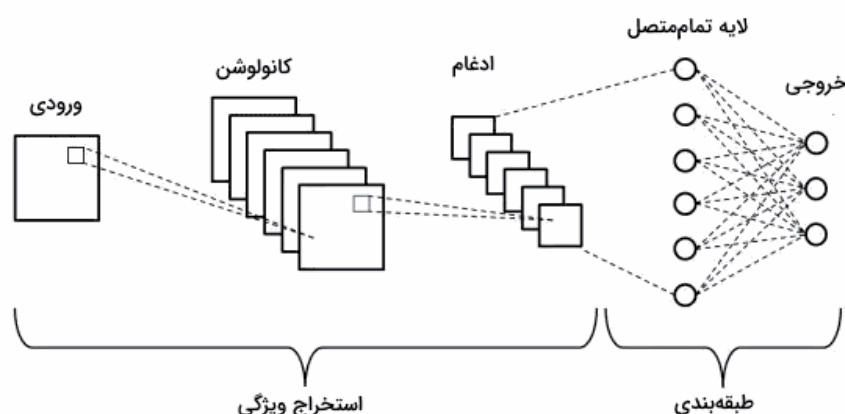
شبکه‌های عصبی پیچشی^{۳۷} یکی از مهم‌ترین معماری‌های یادگیری عمیق در حوزه بینایی ماشین هستند که انقلابی در پردازش تصویر ایجاد کردند [۲۴]. ایده اصلی این شبکه‌ها استفاده از لایه‌های پیچشی است که به جای اتصال کامل نورون‌ها، تنها روی نواحی کوچک و محلی از تصویر عمل می‌کنند. هر فیلتر یا هسته پیچشی با حرکت روی تصویر، الگوهای ابتدایی مانند لبه‌ها یا گوشه‌ها را شناسایی می‌کند و با افزایش عمق شبکه، لایه‌های بعدی می‌توانند ویژگی‌های پیچیده‌تر و انتزاعی‌تر

^{۳۷}Convolutional Neural Networks, CNNs

مانند اشکال یا اشیای کامل را استخراج نمایند.

ویژگی مهم شبکه‌های عصبی پیچشی، اشتراک وزن‌ها است؛ یعنی یک فیلتر در سراسر تصویر اعمال می‌شود و همین موضوع موجب کاهش چشمگیر تعداد پارامترها نسبت به شبکه‌های پیش‌خور می‌گردد. علاوه بر این، استفاده از عملیات زیرنمونه‌برداری (Pooling) مانند ماکس پولینگ باعث کاهش ابعاد و افزایش مقاومت مدل در برابر جابه‌جایی یا تغییرات مکانی می‌شود. ترکیب این ساختار با توابع فعال‌سازی غیرخطی مانند ReLU و روش‌های نرمال‌سازی لایه‌ای، آموزش پایدارتر و همگرایی سریع‌تر را به همراه دارد [۳۲، ۱۸].

موفقیت شبکه‌های عصبی پیچشی با مدل LeNet در تشخیص ارقام دست‌نویس آغاز شد [۳۴] و با معماری‌های عمیق‌تری همچون AlexNet [۳۲]، VGG [۵۳]، Inception [۵۶] و ResNet [۱۸] ادامه یافت. این مدل‌ها توانستند در رقابت‌هایی مانند ImageNet عملکردی فراتر از روش‌های کلاسیک ارائه دهند و استفاده از شبکه‌های عصبی پیچشی را در وظایفی نظیر تشخیص شیء، تقسیم‌بندی معنایی و بینایی رباتیک تثبیت کنند.



شکل ۸.۱: شبکه عصبی پیچشی

با وجود این موفقیت‌ها، شبکه‌های عصبی پیچشی محدودیت‌هایی نیز دارند. نخست آنکه هسته‌های پیچشی ذاتاً بر روابط محلی تمرکز دارند و برای یادگیری وابستگی‌های طولانی در تصویر نیاز به لایه‌های بسیار عمیق است. دوم آنکه افزایش عمق و تعداد پارامترها به‌منظور پوشش بهتر

روابط جهانی موجب هزینه محاسباتی و مصرف حافظه بالایی می‌شود. علاوه بر این، طراحی معماری شبکه‌های عصبی پیچشی غالباً مستلزم تنظیمات دستی متعددی مانند اندازه فیلتر، تعداد لایه‌ها و نحوه اتصال است که فرایند توسعه را دشوار می‌سازد. این محدودیت‌ها توجه پژوهشگران را به سمت معماری‌های نوین جلب کرده است که بتوانند وابستگی‌های بلندبُرد را به‌طور مستقیم و با قابلیت موازی‌سازی کامل مدل کنند. مبدل‌های بینایی^{۳۸} با بهره‌گیری از مکانیزم توجه این کاستی‌ها را جبران کرده و به گزینه‌ای جدی در بینایی ماشین بدل شده‌اند [۱۱].

۱۲.۲.۱ شبکه‌های عصبی بازگشتی و شبکه‌های حافظه بلندمدت کوتاهمدت

شبکه‌های عصبی بازگشتی^{۳۹} و مدل‌هایی با حافظه بلندمدت-کوتاهمدت^{۴۰} با هدف پردازش داده‌های ترتیبی و وابسته به زمان توسعه یافتند [۵۱، ۲۱]. این مدل‌ها به‌ویژه در تحلیل زبان طبیعی، پردازش صوت و پیش‌بینی سری‌های زمانی بسیار موفق عمل کرده‌اند؛ زیرا قادر به حفظ اطلاعات گذشته هستند و از این اطلاعات برای پیش‌بینی در لحظه حال و آینده استفاده می‌کنند [۱۵].

۱۳.۲.۱ شبکه‌های عصبی بازگشتی

مدل‌های اولیه شبکه‌های عصبی، مانند شبکه‌های چندلایه^{۴۱}، برای داده‌های مستقل و با طول ثابت طراحی شده بودند و توانایی مدل‌سازی وابستگی‌های زمانی را نداشتند [۴]. در بسیاری از مسائل دنیای واقعی مانند تحلیل متن، گفتار و سری‌های زمانی، ترتیب داده‌ها و وابستگی بین گام‌های متوالی اهمیت اساسی دارد

Vision Transformers^{۳۸}

RNN^{۳۹}

LSTM^{۴۰}

MLP^{۴۱}

فصل ۲

پیشینه پژوهش

با توجه به مشکلات مطرح شده شبکه‌های بازگشتی، معماری جدیدی به نام مبدل‌ها^۱ مطرح شد. ظهور مبدل‌ها یکی از تحولات اساسی در حوزه پردازش زبان طبیعی^۲ و یادگیری ماشین به شمار می‌رود [۵۹، ۲]. این مدل‌ها باعث تغییرات عمده‌ای در نحوه ساخت و آموزش مدل‌های زبانی و همچنین در بسیاری از کاربردهای دیگر یادگیری ماشین شده‌اند و توانستند بسیاری از مشکلات مدل‌های قبلی را حل کنند [۹، ۵۰].

۱.۲ مشکلات ترجمه ماشینی و مبدل‌ها

در ابتدا، ترجمه ماشینی^۳ یک چالش اساسی در زمینه پردازش زبان طبیعی بود. مدل‌های اولیه‌ای مانند مدل‌های مبتنی بر قواعد^۴ برای ترجمه استفاده می‌شدند که در آن‌ها، ترجمه‌ها به صورت دستی با استفاده از قواعد زبانی مشخص تنظیم می‌شدند [۴۵، ۲۲]. این روش‌ها هرچند دقیق بودند، اما محدودیت‌های زیادی داشتند و نمی‌توانستند ویژگی‌های پیچیده‌تر زبان را مدل‌سازی کنند. سپس مدل‌های آماری^۵ معرفی شدند [۵، ۲۶]. این مدل‌ها از داده‌های ترجمه‌شده برای آموزش

^۱Transformer

^۲NLP

^۳machine translation

^۴Rule-based Models

^۵Statistical Models

مدل‌های آماری استفاده می‌کردند که احتمال ترجمه صحیح را براساس شواهد آماری محاسبه می‌کردند. مدل‌هایی مانند مدل‌های ترجمه آماری مبتنی بر جمله^۶ [۲۷] از این نوع بودند که قادر به ترجمه جملات بهتر از مدل‌های مبتنی بر قواعد بودند، اما هنوز هم در ترجمه‌های پیچیده با مشکلاتی روبه‌رو بودند.

بعد از این مدل‌ها، مدل‌های بازگشتی^۷ به وجود آمدند که مشکلات آن‌ها در فصل گذشته بیان شد [۱۳، ۵۴]. در نهایت، این مشکلات باعث به وجود آمدن ترانسفورمرها شد [۲].

۲.۲ ظهور ترانسفورمرها

در سال ۲۰۱۷، مقاله‌ای توسط گوگل^۸ منتشر شد که مفهوم جدیدی به نام مبدل‌ها^۹ را معرفی کرد [۵۹]. این مقاله به موضوع ترجمه ماشینی پرداخت و نشان داد که با استفاده از مکانیزم توجه می‌توان بسیاری از مشکلات مدل‌های قبلی را حل کرد [۳۷].

مدل‌های ترانسفورمر برخلاف مدل‌های قبلی که از پردازش سریالی استفاده می‌کردند، از پردازش موازی بهره می‌برند. این ویژگی به ترانسفورمرها اجازه می‌دهد که به‌طور همزمان به تمام بخش‌های ورودی توجه کنند. این قابلیت باعث شد که مبدل‌ها در پردازش تصویر و متن بسیار سریع‌تر و دقیق‌تر از مدل‌های قبلی عمل کنند [۵۹].

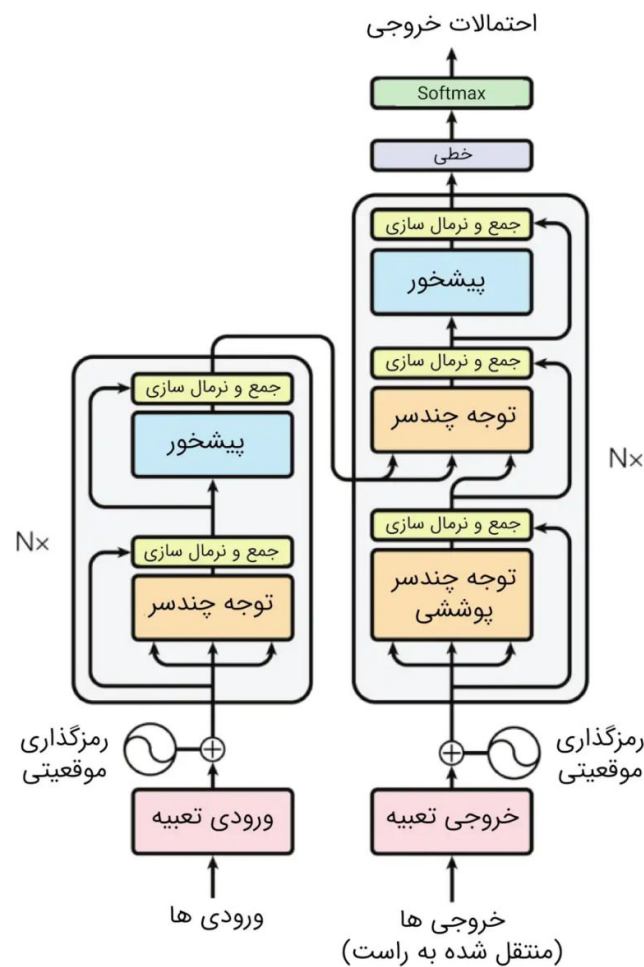
۳.۲ معماری مبدل‌ها

در تصویر ۱.۲، معماری مبدل نمایش داده شده است و بخش‌ها و اجزای مختلف آن مشخص شده است. معماری ترانسفورمر از دو بخش اصلی تشکیل شده است:

Phrase-based Statistical Models^۶
 Recurrent Models^۷
 google^۸
 Transformers^۹

رمزگذار^{۱۰} وظیفه رمزگذار این است که داده ورودی را دریافت کند و ویژگی‌های آن را استخراج کند.

رمزگشا^{۱۱} وظیفه رمزگشا این است که ویژگی‌های استخراج‌شده را به زبان مقصد تبدیل کند.



شکل ۱۰۲: معماری ترانسفورمرها

^{۱۰} Encoder

^{۱۱} Decoder

۱.۳.۲ جاسازی

در زبان طبیعی، کلمات به شکل رشته‌های متنی هستند مانند کتاب، ماشین و ... کامپیوترها نمی‌توانند به‌طور مستقیم این کلمات را به شکل رشته‌های متنی پردازش کنند. به همین دلیل، در یادگیری ماشین این کلمات را به شکل یک بردار نمایش می‌دهیم. این بردار بیانگر آن کلمه در مدل است تا ماشین بتواند آن کلمه را پردازش کند.

این بردارها ویژگی‌های کلمه را در فضای عددی نمایش می‌دهند. روش‌های مختلفی برای تبدیل متن به بردار وجود دارند. از جمله این روش‌ها می‌توان به روش‌های Word2Vec [۴۱] و GloVe [۴۹] اشاره کرد.

همان‌طور که در شکل ۲.۲ نشان داده شده است، هر کلمه که به‌صورت توکن است، ابتدا در دیکشنری تعریف شده پیدا می‌شود و پس از پیدا شدن در دیکشنری، با استفاده از روش‌های تعبیه کردن^{۱۲}، هر کلمه به برداری از اعداد تبدیل می‌شود. این جاسازی‌ها شباهت‌های معنایی بین کلمات را مدل‌سازی می‌کنند و کلماتی که از نظر معنایی شبیه به هم هستند، بردار آن‌ها نیز به یکدیگر نزدیک‌تر است. به این ترتیب، کلمات برای مدل‌ها و شبکه‌های عصبی قابل فهم می‌شوند [۴۱، ۴۹].

بود	گرم	و	آفتابی	هوا	امروز
105	6587	5475	3578	65	6587
952.207	171.411	621.659	776.562	6422.693	171.411
5450.840	3276.350	1304.051	5567.288	6315.080	3276.350
1853.448	9192.819	0.565	58.942	9358.778	9192.819
...	...	...	...	...	...
1.658	3633.421	7679.805	2716.194	2141.081	3633.421
2671.529	8390.473	4506.025	5119.949	735.147	8390.473

شکل ۲.۲: جاسازی کلمه‌ای

۲.۳.۲ جاسازی موقعیتی

هر کلمه را به برداری از اعداد که برای مدل قابل فهم باشد، تبدیل کرده‌ایم. اما مدل‌های ترانسفورمر نمی‌توانند جایگاه هر کلمه را تشخیص دهند. در مدل‌های ترانسفورمر، برخلاف مدل‌های بازگشتی، به دلیل اینکه کلمات به صورت موازی وارد می‌شوند، نیاز داریم تا جایگاه هر کلمه را بدانیم. به طور مثال، در جمله «من تو را دوست دارم» باید به طور دقیق بدانیم که «من» کلمه اول جمله است، «تو» کلمه دوم جمله است و

حال باید به مدل توالی این کلمات را بفهمانیم. بنابراین، نیاز داریم به مدل یک سری اطلاعات اضافی بدهیم به طوری که مدل توالی کلمات را یاد بگیرد. روش‌های مختلفی برای اضافه کردن جاسازی موقعیتی^{۱۳} به مدل وجود دارد. در ترانسفورمرها از روش جاسازی موقعیت سینوسی^{۱۴} استفاده می‌شود [۵۹].

این روش قابل یادگیری نیست و صرفاً از یک سری فرمول‌های ساده برای جاسازی موقعیتی استفاده می‌کند. برای موقعیت مشخص^{۱۵} در توالی و بعد i در فضای برداری، تعبیه موقعیتی به صورت زیر تعریف می‌شود.

رابطه برای مقادیر زوج:

$$PE(pos, 2i) = \sin\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (1.2)$$

و برای مقادیر فرد:

$$PE(pos, 2i + 1) = \cos\left(\frac{pos}{10000^{\frac{2i}{d}}}\right) \quad (2.2)$$

• pos : موقعیت کلمه در توالی است (مثلاً از 0 تا $N - 1$ برای یک توالی N -تایی).

• i : شاخص بعد در بردار موقعیتی (از 0 تا $d - 1$ برای بعد فضای برداری d).

^{۱۳} Positional Embedding

^{۱۴} Sinusoidal Positional Embedding

^{۱۵} pos

- d : ابعاد فضای برداری مدل که نشان می‌دهد هر کلمه در چند بعد نمایش داده می‌شود.
- 10000: یک مقدار ثابت برای تنظیم مقیاس توابع تناوبی و ایجاد فرکانس‌های مختلف در ابعاد گوناگون.

همان‌طور که در شکل **۳.۲** مشاهده می‌کنید، بعد از جاسازی کلمات، به آن جاسازی موقعیتی اضافه می‌شود. در این روش از توابع سینوس و کسینوس استفاده می‌شود. این توابع موقعیت‌ها را در فضای برداری به گونه‌ای نگاشت می‌کنند که مدل بتواند از ترتیب کلمات در توالی آگاه باشد [۵۹]. این ویژگی به مدل کمک می‌کند تا توالی زمانی را درک کرده و الگوهای زمانی را شبیه‌سازی کند. از مزایای این روش می‌توان به عدم نیاز به آموزش و توزیع متوازن جایگاه کلمات اشاره کرد.

بود	گرم	و	آفتابی	هوا	امروز
952,207	171,411	621,659	776,562	6422,693	171,411
5450,840	3276,350	1304,051	5567,288	6315,080	3276,350
1853,448	9192,819	0,565	58,942	9358,778	9192,819
...	...	...	...	...	...
1,458	3633,421	7679,805	2716,194	2141,081	3633,421
2671,529	8395,473	8395,025	5119,949	735,147	8395,473
+	+	+	+	+	+
...	1664,088	...	...	...	1281,458
...	8080,133	...	...	...	7902,890
...	2820,399	...	...	...	6023,970
...	9384,405	...	...	...	3821,102
...	...	...	...	...	1689,217
...	8130,159	...	...	...	7018,630

شکل ۳.۲: اضافه کردن جاسازی موقعیتی به جاسازی کلمه‌ای

۳.۳.۲ توجه

در روش شبکه‌های بازگشتی، توالی ورودی (مثلاً یک جمله) معمولاً به صورت گام به گام پردازش می‌شد [۲۱، ۱۳]. اما در ترانسفورمر می‌خواهیم مدلی داشته باشیم که به هر موقعیت (مثلاً یک کلمه) در توالی نگاه کند و به همه موقعیت‌های دیگر نیز به صورت موازی دسترسی داشته باشد. به این مفهوم توجه^{۱۶} می‌گوییم.

^{۱۶}attention

به زبان ساده، وقتی توکن (کلمه) i به توکن‌های دیگر نگاه می‌کند، می‌خواهد بداند کدام توکن‌ها برای تفسیر معنای خودش مهم‌ترند.

به طور مثال در جمله «یک گربه روی زمین نشسته است» می‌خواهد بداند کلمه «گربه» به واژه «نشستن» بیشتر توجه کند یا به «زمین». در این جا فعل «نشستن» ارتباط نزدیک‌تری به «گربه» دارد و از نظر معنایی مرتبط‌تر است.

سه تا از اجزای اصلی یک توجه شامل موارد زیر است.

Value (مقدار / ارزش) V , Key (کلید) K , Query (پرسش) Q

در ضرب شباهت‌های توجه ^{۱۷} [۵۹]، ابتدا شباهت یا ارتباط بین پرسش ^{۱۸} و کلید ^{۱۹} را با محاسبه ضرب داخلی ^{۲۰} به دست می‌آوریم، سپس آن را نرمال می‌کنیم (با تقسیم بر d_k) و از تابع سافت مکس ^{۲۱} استفاده می‌کنیم تا ضرایب توجه ^{۲۲} را به دست آوریم. در نهایت با همین ضرایب، ترکیبی خطی از بردارهای مقدار ^{۲۳} را می‌گیریم.

فرمول به شکل زیر است:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (۳.۲)$$

که در آن:

ماتریس پرسش برای $Q \in \mathbb{R}^{n \times d_k}$

^{۱۷} Scaled Dot-Product Attention

^{۱۸} Query

^{۱۹} Key

^{۲۰} Dot Product

^{۲۱} softmax

^{۲۲} Attention Weights

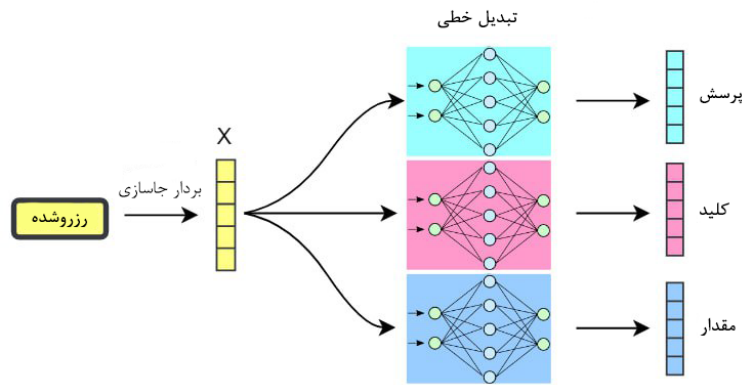
^{۲۳} value

ماتریس کلید برای $K \in \mathbb{R}^{n \times d_k}$

ماتریس مقدار $V \in \mathbb{R}^{n \times d_v}$

$W^Q, W^K, W^V \in \mathbb{R}^{d_{model} \times d_k}$ ماتریس‌های وزنی قابل آموزش هستند که طی فرآیند یادگیری

به‌روزرسانی می‌شوند.



شکل ۴.۲: نحوه به دست آوردن پرسش، کلید و مقدار

در واقع پرسش، کلید و مقدار با استفاده از mlp تولید می‌شود.

تقسیم بر d_k باعث می‌شود مقدار ضرب داخلی در ابعاد بالا خیلی بزرگ نشود و شیب‌ها گرادینان پایدار بمانند.

$$\alpha = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) \quad (۴.۲)$$

α یک ماتریس با ابعاد $n \times n$ است که سطر i -ام آن ضرایب توجه برای توکن i را نشان می‌دهد. تفسیر ضرایب توجه: هر سطر از α نشان می‌دهد که توکن فعلی به چه توکن‌هایی در جمله، با چه شدتی توجه می‌کند.

	امروز	هوا	آفتابی	و	گرم	بود
امروز	0.152	0.179	0.148	0.134	0.119	0.268
هوا	0.115	0.154	0.128	0.201	0.278	0.124
آفتابی	0.145	0.218	0.097	0.262	0.132	0.147
و	0.125	0.119	0.212	0.206	0.128	0.210
گرم	0.174	0.227	0.143	0.152	0.158	0.146
بود	0.229	0.157	0.103	0.203	0.114	0.195

شکل ۵.۲: توجه

۴.۳.۲ توجه چندسر

در ایده چندسری^{۲۴} به جای آنکه فقط یک بار Q, K, V بسازیم و عملیات توجه را انجام دهیم، چندین مجموعه متفاوت Q_i, K_i, V_i می‌سازیم (هر کدام یک «سر»^{۲۵} نام دارد) و به صورت موازی محاسبات توجه را انجام می‌دهیم. سپس خروجی همه این سرها را کنار هم قرار داده^{۲۶} و در نهایت با یک ماتریس وزن دیگر ضرب می‌کنیم تا به بعد اصلی بازگردیم.

فرمول مربوط به این ایده به شکل زیر است:

$$\text{head}_i = \text{Attention}(Q_i, K_i, V_i) \quad (۵.۲)$$

multi head attention^{۲۴}

head^{۲۵}

concatenate^{۲۶}

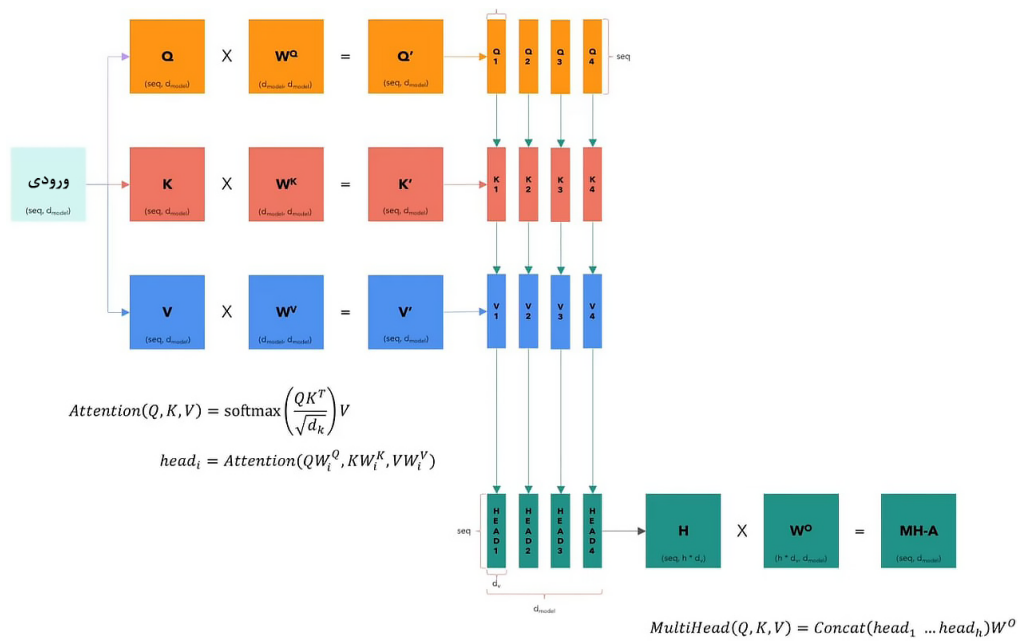
$$\text{MultiHead}(Q, K, V) = [\text{head}_1 \oplus \dots \oplus \text{head}_h] W_O \quad (۶.۲)$$

که در آن $\oplus$ نشان‌دهنده عمل الحاق^{۲۷} است.

ماتریس وزن W_O به شکل زیر است:

$$W_O \in \mathbb{R}^{(h \cdot d_v) \times d_{\text{model}}}$$

که W_O ماتریسی است که خروجی الحاق شده را به بعد d_{model} برمی گرداند.



شکل ۶.۲: توجه چندسر

چرا چندین سر؟

مشاهده چند منظر متفاوت: هر سر می تواند الگوهای گوناگونی از وابستگی ها را بیاموزد (مثلاً یک سر می تواند یاد بگیرد کلمه فعلی با کلمات همسایه نزدیک خود بیشتر مرتبط شود، یک سر دیگر

^{۲۷} concatenate

روی ارتباط با کلماتی در فاصله دورتری متمرکز باشد، سر دیگر روی مطابقت جنس و تعداد در دستور زبان و ...).

افزایش ظرفیت مدل: با داشتن چند سر، مدل می‌تواند قدرت بیان بیشتری داشته باشد. ابعاد کمتر در هر سر: در عمل، اگر d_{model} مثلاً ۵۱۲ باشد، و تعداد سرها $h = 8$ ، آنگاه هر سر ابعادی در حدود $d_k = 64$ خواهد داشت؛ و این محاسبات ضرب داخلی را نیز مقیاس پذیر و قابل موازی‌سازی می‌کند.

۵.۳.۲ اتصال باقیمانده

در معماری‌های عمیق، هنگامی که تعداد لایه‌ها زیاد می‌شود، اغلب دچار ناپایداری گرادیان می‌شوند و این مشکل باعث دشواری در آموزش مدل می‌گردد [۲۱، ۳].

در مبدل‌ها [۵۹]، به جای این که خروجی توجه را به صورت مستقیم به لایه بعدی بدهیم، ورودی آن را نیز حفظ کرده و به خروجی اضافه می‌کنیم. ایده اصلی این روش از اتصالات باقی مانده^{۲۸} در شبکه‌های عمیق الهام گرفته شده است [۱۸].

اگر x ورودی به زیرماژول و $\text{SubLayer}(x)$ خروجی آن زیرماژول باشد، در انتهای کار عبارت زیر را محاسبه می‌کنیم:

$$x + \text{SubLayer}(x) \quad (۷.۲)$$

این جمع به صورت عنصر به عنصر^{۲۹} انجام می‌شود.

اتصال باقیمانده در مبدل‌ها چندین مزیت دارد که عبارت‌اند از:

^{۲۸} Residual Connection
^{۲۹} Element-wise Addition

کمک به جریان یافتن گرادیان

وقتی ورودی مستقیماً به خروجی اضافه می‌شود، مسیری مستقیم برای عبور شیب (گرادیان) به عقب ایجاد می‌گردد. در صورت نبود این اتصال، اگر شبکه عمیق شود، گرادیان‌ها ممکن است در لایه‌های پایین محو شوند و عملاً ناپدید شدن گرادیان^{۳۰} رخ دهد [۲۱، ۳].

حفظ اطلاعات اصلی (هویت ورودی)

حتی اگر زیرماژول تغییری در اطلاعات ورودی ایجاد کند، با وجود اتصال باقیمانده^{۳۱}، ورودی اصلی همواره در خروجی نهایی حضور دارد. این ویژگی باعث می‌شود در صورت ناکافی بودن یادگیری زیرماژول یا در مراحل اولیه آموزش، دست‌کم بخشی از سیگنال (اطلاعات) خام به لایه‌های بالاتر برسد [۱۸، ۵۹].

کاهش ریسک تخریب ویژگی‌ها

در شبکه‌های عمیق، یکی از مشکلات این است که هر لایه ممکن است بخشی از اطلاعات مفید را تخریب کند. اتصال باقیمانده تضمین می‌کند که اگر لایه‌ای به هر دلیل نتوانست الگوی بهینه را یاد بگیرد، اطلاعات قبلی حداقل به صورت دست‌نخورده تا حدی منتقل می‌شود.

۴.۲ نرمال‌سازی لایه‌ها

در یادگیری عمیق، نرمال‌سازی^{۳۲} داده‌های یک لایه یا فعال‌سازی‌ها، اغلب به سرعت بخشیدن به همگرایی و پایدار کردن آموزش کمک شایانی می‌کند. شاید معروف‌ترین نوع نرمال‌سازی، نرمال‌سازی بچ^{۳۳} باشد که پیش‌تر در کارهای بینایی کانولوشنی^{۳۴} بسیار مورد استفاده قرار گرفت

^{۳۰} gradient vanishing

^{۳۱} Residual Connection

^{۳۲} Normalization

^{۳۳} Batch Normalization

^{۳۴} CNN

[۲۳].

نرمال سازی لایه ها^{۳۵} روشی جایگزین است که در ترانسفورمر استفاده می شود [۱، ۵۹]. علت اصلی این انتخاب، ماهیت توالی محور^{۳۶} بودن داده ها در پردازش زبان طبیعی و عدم تمایل به وابستگی به آمار مینی بچ^{۳۷} است.

نرمال سازی بچ

در نرمال سازی بچ ها، برای نرمال سازی، میانگین و واریانس روی تمام نمونه های موجود در مینی بچ (و نیز در طول ابعاد ویژگی) محاسبه می شود [۲۳]. این موضوع در پردازش زبان طبیعی کمی در دسرساز است؛ چون ترتیب توکن ها، طول جمله ها و حتی اندازه مینی بچ ممکن است نامنظم باشد. همچنین به خاطر تنوع طول توالی ها، پیاده سازی نرمال سازی بچ ها می تواند پیچیده شود.

نرمال سازی لایه ها

در نرمال سازی لایه ها، برای هر توکن به صورت جداگانه (در طول بعد ویژگی)، میانگین^{۳۸} و واریانس^{۳۹} گرفته می شود [۱]. فرض کنید در یک لایه، بردار $h_i \in \mathbb{R}^{d_{\text{model}}}$ مربوط به توکن i باشد؛ یعنی ابعاد ویژگی آن d_{model} است. ما میانگین μ_i و واریانس σ_i^2 را از اجزای این بردار محاسبه می کنیم:

$$\mu_i = \frac{1}{d_{\text{model}}} \sum_{k=1}^{d_{\text{model}}} h_{i,k}, \quad \sigma_i^2 = \frac{1}{d_{\text{model}}} \sum_{k=1}^{d_{\text{model}}} (h_{i,k} - \mu_i)^2 \quad (۸.۲)$$

سپس نرمال سازی برای هر مؤلفه k در بردار توکن i به شکل زیر انجام می شود:

$$\hat{h}_{i,k} = \frac{h_{i,k} - \mu_i}{\sqrt{\sigma_i^2 + \epsilon}} \quad (۹.۲)$$

Layer Normalization^{۳۵}

sequence^{۳۶}

mini-batch^{۳۷}

mean^{۳۸}

variance^{۳۹}

در نهایت، برای این که مدل بتواند مقیاس و بایاس جدیدی یاد بگیرد، شبیه بچ نرم، دو پارامتر γ و β نیز در طول بعد ویژگی اعمال می شوند:

$$\text{LayerNorm}(h_i) = \gamma \odot \hat{h}_i + \beta \quad (10.2)$$

که در آن $\gamma, \beta \in \mathbb{R}^{d_{\text{model}}}$ هستند و $\odot$ ضرب عنصر به عنصر است [۱].

مزایای نرمال سازی لایه در مبدل ها

- بی نیازی از وابستگی به ابعاد مینی بچ: با نرمال سازی لایه، می توان حتی با اندازه مینی بچ برابر ۱ نیز به خوبی آموزش دید، چرا که آمارها وابسته به ابعاد ویژگی اند و نه مینی بچ [۱].
 - پایداری سازی توزیع فعال سازی ها: زمانی که مدل در حال یادگیری است، توزیع های داخلی لایه های میانی ممکن است تغییر کند.^{۴۰} نرمال سازی لایه با نرمال سازی این توزیع، آموزش را پایدارتر و سریع تر می کند [۱، ۲۳].
 - سازگاری با داده های توالی محور: هر توکن را جداگانه نرمال می کند و نگرانی ای بابت ترتیب طول جمله ها، یا قرار گرفتن چند جمله کوتاه/بلند در یک مینی بچ نداریم [۵۹].
- در معماری مبدل ها، پس از خروجی هر زیرماژول، مراحل به شکل زیر است:

۱.۴.۲ اتصال باقیمانده

ابتدا ورودی همان زیرماژول (مثلاً بردار x) را با خروجی زیرماژول ($\text{SubLayer}(x)$) جمع می کنیم. حاصل این جمع را می توان چنین نوشت:

$$z = x + \text{SubLayer}(x) \quad (11.2)$$

Internal Covariate Shift^{۴۰}

این z حالا ترکیبی از اطلاعات اصلی ورودی و اطلاعات یادگرفته شده توسط SubLayer است.

نرمال سازی لایه سپس این بردار z را وارد لایه LayerNorm می کنیم.

$$y = \text{LayerNorm}(z)$$

خروجی نهایی را می توان به لایه بعدی پاس داد یا به مرحله بعدی در همین لایه.

به عبارتی اگر بخواهیم در یک فرمول واحد بیان کنیم:

$$\text{Norm \& Add} = \text{LayerNorm}(x + \text{SubLayer}(x))$$

۵.۲ رمزگشا

رمزگشا در معماری ترانسفورمرها وظیفه تولید خروجی نهایی را بر عهده دارد. این خروجی معمولاً می تواند توالی هدف^{۴۱} باشد، مانند ترجمه یک جمله یا پیش بینی توکن های بعدی در یک توالی [۵۹].

در این بخش، رمزگشا دو ورودی اصلی دارد: ۱. توالی هدف که معمولاً به صورت خودکار تولید می شود (مثلاً در ترجمه ماشینی یا تولید متن)، ۲. نمایش اطلاعات گذشته که توسط انکودر تولید شده است و شامل ویژگی های استخراج شده از توالی ورودی می باشد.

رمزگشا از این ورودی ها استفاده می کند تا به صورت گام به گام، خروجی نهایی خود را تولید کند

[۵۴، ۲].

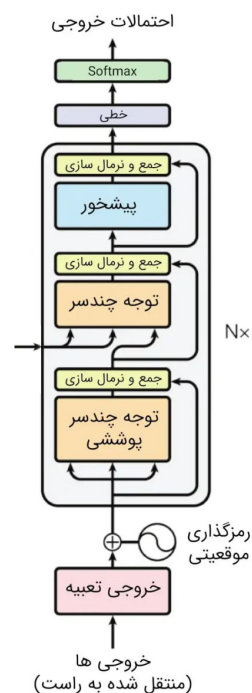
همان طور که در شکل ۷.۲ مشاهده می کنید، رمزگشا دو ورودی دارد.

تمامی بخش های رمزگشا مانند رمزگذار هستند اما در دیکودر توجه چند سر ماسک شده^{۴۲} وجود

دارد [۵۹].

^{۴۱} Target Sequence

^{۴۲} Masked Multi-Head Attention



شکل ۷.۲: رمزگشا در مبدل‌ها

۶.۲ توجه چندسری ماسک‌شده

در مبدل‌ها، مکانیزم توجه چندسری^{۴۳} در بخش دیکودر به صورت ماسک‌شده^{۴۴} پیاده‌سازی می‌شود تا مدل نتواند توکن‌های آینده را ببیند و به صورت خودبازگشتی^{۴۵} توکن بعدی را پیش‌بینی کند [۵۹]. در واقع ایده اصلی استفاده از ماسک جلوگیری از مشاهده آینده است.

در معماری‌های خودبازگشتی، مدل در گام i از دیکودر تنها باید به توکن‌های قبلی $\{y_1, \dots, y_{i-1}\}$ دسترسی داشته باشد؛ اما نه به توکن‌های $\{y_{i+1}, y_{i+2}, \dots\}$. اگر مدل بتواند توکن‌های آینده را «نگاه» کند، پیش‌بینی توکن بعدی آسان و غیرواقعی می‌شود (مشکل نشت اطلاعات) [۲، ۵۴].

به همین دلیل در توجه چندسری ماسک‌شده در دیکودر، از یک ماتریس ماسک M استفاده می‌کنیم که اجازه نمی‌دهد هر توکن به توکن‌های آینده‌اش توجه کند.

^{۴۳} Multi-Head Attention

^{۴۴} Masked

^{۴۵} Autoregressive

۷.۲ مثال عددی توجه ماسک شده

فرض کنید دنباله‌ای چهارتوکنی به صورت

$$[y_1, y_2, y_3, y_4]$$

داشته باشیم. خروجی ضرب داخلی (قبل از اعمال softmax) یک ماتریس 4×4 خواهد بود:

$$S = \begin{bmatrix} s_{1,1} & s_{1,2} & s_{1,3} & s_{1,4} \\ s_{2,1} & s_{2,2} & s_{2,3} & s_{2,4} \\ s_{3,1} & s_{3,2} & s_{3,3} & s_{3,4} \\ s_{4,1} & s_{4,2} & s_{4,3} & s_{4,4} \end{bmatrix}$$

در این ماتریس، هر سطر نشان‌دهنده دسترسی یک توکن به سایر توکن‌ها در مکانیزم توجه ماسک شده است. در سطر اول، توکن نخست تنها می‌تواند به خودش (ستون ۱) توجه کند و ستون‌های ۲ تا ۴ ماسک می‌شوند. در سطر دوم، توکن دوم به ستون‌های ۱ و ۲ دسترسی دارد، در حالی‌که ستون‌های ۳ و ۴ ماسک می‌شوند. در سطر سوم، توکن سوم می‌تواند به ستون‌های ۱، ۲ و ۳ توجه کند اما ستون چهارم برای آن ماسک می‌شود. در نهایت، سطر چهارم نشان می‌دهد که توکن چهارم قادر است به همه ستون‌های ۱ تا ۴ دسترسی داشته باشد. به عبارت دیگر، آخرین توکن می‌تواند تمام توکن‌های قبلی و معمولاً خودش را نیز ببیند. البته بسته به پیاده‌سازی، ممکن است توکن فعلی از توجه به خودش محروم شود، اما در معماری استاندارد معمولاً سطر i به ستون i نیز دسترسی دارد.

در عمل، ماتریس ماسک M به شکل زیر خواهد بود (با نشانه‌گذاری پایین‌مشی):

$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

به این ترتیب، پس از جمع شدن با S و اجرای softmax در هر سطر، ضرایب توجه ستون‌های ماسک شده به صفر میل می‌کنند [۵۹].

۸.۲ مبدل‌های بینایی

ایده مبدل‌ها در حوزه بینایی^{۴۶} از تعمیم مبدل متن به تصاویر به وجود آمده است [۱۱]. ما در این بخش از مبدل‌های بینایی برای وظیفه کلاس‌بندی استفاده می‌کنیم. در روش‌های متداول برای پردازش تصویر، از شبکه‌های پیچشی^{۴۷} های متوالی استفاده می‌کردند؛ اما در ترانسفورمرها تصاویر به پچ‌های مختلف شکسته می‌شوند [۱۱]. هر پچ شکسته شده از تصویر می‌تواند با سایر پچ‌ها به صورت موازی وارد مکانیزم توجه شود و شباهت یا ارتباطشان با یکدیگر سنجیده شود. در بخش‌های بعد، به طور مفصل روند انجام این کار را توضیح خواهیم داد.

۱.۸.۲ جاسازی پچ‌ها در مبدل‌های بینایی

در مبدل مبتنی بر متن، هر کلمه به توکن تبدیل می‌شود و سپس هر کلمه به برداری تبدیل می‌گردد. این بردارها پس از افزودن جاسازی موقعیتی وارد مکانیزم توجه می‌شوند [۵۹]. حال همین ایده در تصویر پیاده‌سازی شده است. همان‌طور که در شکل ۸.۲ مشاهده می‌کنید، در مبدل‌های بینایی، به جای استفاده از عملیات کانولوشن‌های متوالی که در شبکه‌های پیچشی^{۴۸} مرسوم است [۱۸، ۳۲، ۳۴]، تصویر را به بلاک‌های غیرهم‌پوشان $(P \times P)$ تقسیم می‌کنیم. این کار علاوه بر ساده‌سازی موازی‌سازی، به مدل اجازه می‌دهد از سازوکار توجه برای ارتباط بین این بلاک‌ها استفاده کند [۱۱].

^{۴۶} vision transformer

^{۴۷} convolution

^{۴۸} convolutional neural network



شکل ۸.۲: بخش بندی تصاویر

۲.۸.۲ شکل پچ‌ها:

فرض کنید ابعاد تصویر ورودی $(H \times W \times C)$ باشد. به عنوان مثال، اگر اندازه تصویر $224 \times 224 \times 3$ باشد، طول و عرض تصویر به ترتیب 224 و تصویر دارای سه کانال رنگی است:

$$H = 224, \quad W = 224, \quad C = 3$$

حال اگر اندازه هر پچ $(P \times P)$ باشد (برای نمونه 16×16)، تصویر به صورت یک جدول مشبک از پچ‌های کوچک تقسیم می‌شود. به هر پچ می‌توان مانند یک «کاشی» از تصویر نگاه کرد: - پچ اول: مختصات (در ارتفاع 15 تا 0) و (در عرض 15 تا 0)، - پچ دوم: مختصات (در ارتفاع 15 تا 0) و (در عرض 31 تا 16)، - و به همین ترتیب تا کل تصویر پوشش داده شود.

۳.۸.۲ تعداد پچ‌ها:

اگر پچ‌ها بدون هم‌پوشانی باشند، ابعاد پچ باید بر ابعاد تصویر بخش پذیر باشد.

$$\frac{H}{P} - \text{تعداد پچ‌های افقی:} \quad \frac{W}{P} - \text{تعداد پچ‌های عمودی:} \quad \frac{H}{P}$$

در مجموع:

$$\left(\frac{H}{P}\right) \times \left(\frac{W}{P}\right) = \frac{H}{P} \times \frac{W}{P}. \quad (12.2)$$

برای مثال اگر:

$$H = 224, \quad W = 224, \quad P = 16 :$$

$$\frac{224}{16} = 14 \Rightarrow 14 \times 14 = 196 \quad (\text{تعداد پچ‌ها}).$$

در اکثر نسخه‌های مبدل‌های بینایی، پچ‌ها بدون هم‌پوشانی^{۴۹} هستند. اندازه پچ‌های کوچک باعث می‌شود تعداد پچ‌ها زیاد شود و در نتیجه هزینه توجه بالا رود. از طرفی، پچ‌های بزرگ هزینه توجه را کاهش می‌دهند؛ اما ممکن است جزییات محلی^{۵۰} را از دست بدهیم [۱۱].

۴.۸.۲ بردار کردن هر پچ

هر پچ دارای ابعاد $(P \times P \times C)$ است. برای مثال اگر $P = 16$ و $C = 3$ ، آنگاه پچ ابعاد $16 \times 16 \times 3$ خواهد داشت. برای این‌که بتوانیم پچ‌ها را مانند «توکن»های پردازش زبان طبیعی به مبدل‌ها بدهیم، باید آن‌ها را به یک بردار یک‌بعدی تبدیل کنیم. در صورت قرار دادن پیکسل‌های پچ به صورت ردیفی^{۵۱}، طول این بردار خواهد بود:

$$P \times P \times C = P^2 \times C. \quad (۱۳.۲)$$

در مثال $(16 \times 16 \times 3)$ ، طول بردار می‌شود 768.

۹.۲ اعمال لایه خطی

بعد از کنار هم چیدن پچ‌ها^{۵۲}، معمولاً یک لایه خطی^{۵۳} روی این بردار اعمال می‌شود تا آن را به بعد d_{model} (مثلاً ۷۶۸ یا ۱۰۲۴) برسد. در حقیقت، این لایه یک تبدیل ویژگی^{۵۴} انجام می‌دهد تا همه پچ‌ها یک نمایندگی (تعبیه‌شده) با ابعاد یکنواخت d_{model} پیدا کنند:

$$(P^2 \times C) \rightarrow d_{\text{model}}$$

^{۴۹}Non-overlapping

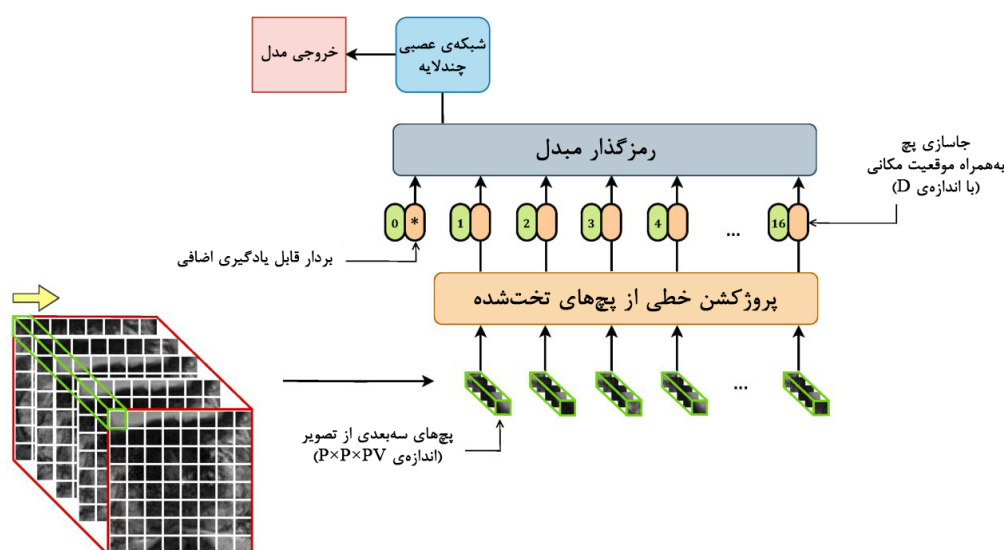
^{۵۰}Local Details

^{۵۱}Row-major

^{۵۲}Flatten

^{۵۳}Fully-Connected Layer

^{۵۴}Feature Transformation



شکل ۹.۲: مبدل‌های بینایی

این مرحله شبیه ساخت توکن در پردازش زبان طبیعی است؛ با این تفاوت که در پردازش زبان طبیعی، توکن «کلمه» یا «زیرکلمه» است و از قبل دارای بردار تعبیه‌شده جاساز شده بوده است [۵۹]. در مبدل‌های بینایی [۱۱]، ما ابتدا باید تصاویر را پچ کنیم و سپس بردارهای جاساز را از این پچ‌ها به دست آوریم.

مبدل نیاز دارد ورودی‌اش توالی توکن‌ها باشد. در پردازش زبان طبیعی توالی کلمات داریم، در مبدل‌های بینایی توالی «پچ»‌ها:

$$\{x_{\text{patch}_1}, x_{\text{patch}_2}, \dots, x_{\text{patch}_N}\}.$$

هر پچ اکنون یک بردار d_{model} -بعدی است. پس یک مجموعه با طول N (تعداد پچ‌ها) و عرض d_{model} خواهیم داشت. اگر عدد پچ‌ها N باشد (مثلاً ۱۹۶)، ترانسفورمر می‌تواند با مکانیزم توجه خود سر، وابستگی میان پچ‌ها را یاد بگیرد: کدام بخش از تصویر برای کدام بخش دیگر مهم‌تر است [۵۹، ۱۱].

معمولاً پچ‌ها را به صورت ردیفی شماره‌گذاری می‌کنند (ابتدا پچ‌های ردیف بالایی از چپ به راست، سپس ردیف بعدی و ...)، تا مدل در صورت نیاز بتواند از موقعیت‌ها، اطلاعات مکانی

تقریبی داشته باشد. در عمل، چون قصد داریم (در مراحل بعد) به هر پچ یک جاسازی موقعیتی هم اضافه کنیم، مکان دقیق هر پچ در بعد دوم (ویژگی) کد می‌شود. در مبدل بینایی [۱۱] دیگر به کانولوشن وابسته نیستیم. در عوض، از جاسازی استفاده می‌شود. تقسیم‌کردن تصویر به بلاک‌های $(P \times P)$ ، کنار هم چیدن و تبدیل آن به جاساز همگی عملیات ریاضی ساده‌ای هستند که به راحتی روی TPU/GPU قابل موازی‌سازی‌اند.

۱.۹.۲ توکن کلاس‌بندی

توکن کلاس‌بندی^{۵۵} یک بردار ویژه است که به ابتدای دنباله ورودی اضافه می‌شود و نقش آن، خلاصه‌کردن اطلاعات کل ورودی (چه متن، چه تصویر) است [۹، ۱۱].

در مبدل بینایی، این توکن در ابتدای پچ‌های تصویری قرار می‌گیرد. این توکن یک بردار با ابعاد d_{model} است (همان ابعاد سایر توکن‌ها) و پارامتری یادگرفته محسوب می‌شود؛ یعنی مدل طی آموزش، مقادیر آن را برای ذخیره و تجمیع اطلاعات بهینه می‌کند.

در وظایف دسته‌بندی کلاس‌بندی، هدف این است که یک پیش‌بینی کلی برای کل ورودی (مثلاً یک جمله یا یک تصویر) ارائه دهیم؛ توکن کلاس‌بندی دقیقاً همین وظیفه را بر عهده دارد [۹]. این توکن از طریق مکانیزم توجه چندسر در مبدل‌ها با تمامی توکن‌های دیگر (پچ‌های تصویر) ارتباط می‌گیرد و اطلاعات مهم آن‌ها را در لایه‌های مختلف مبدل‌ها را به صورت تجمعی یاد می‌گیرد. به عبارتی، توکن کلاس‌بندی نقش نماینده کل تصویر یا متن را بر عهده دارد.

توکن کلاس‌بندی از طریق ضرب داخلی در مکانیزم توجه، می‌تواند به تمام پچ‌ها نگاه کند و با ضرایب توجه (α) مشخص کند که از هر پچ چه مقدار اطلاعات بگیرد. بدین ترتیب، به طور ضمنی یاد می‌گیرد روی ویژگی‌هایی که برای دسته‌بندی مهم هستند (نظیر الگوها، اشکال و بخش‌های کلیدی تصویر) متمرکز شود.

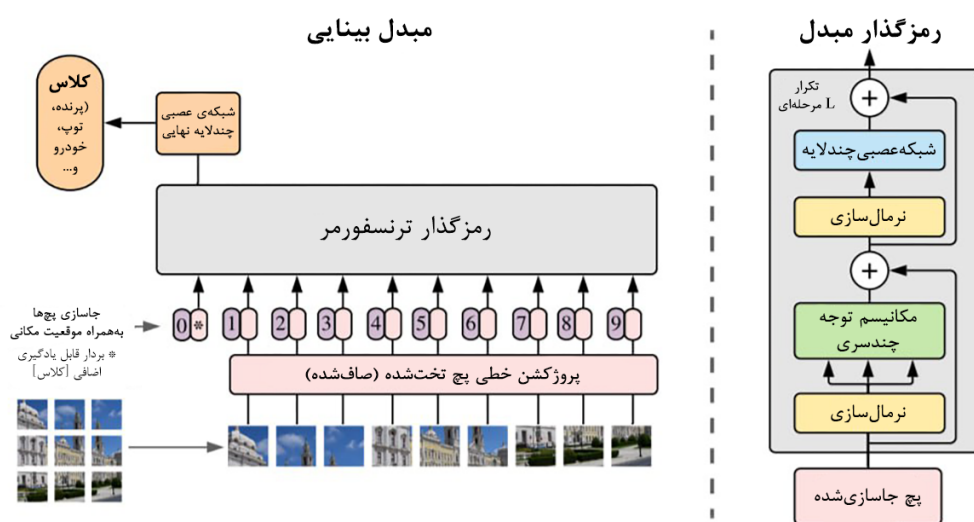
در طول لایه‌های ترانسفورمر، توکن کلاس‌بندی نقش محوری در خلاصه‌سازی بازنمایی کل

^{۵۵} Cls Token

تصویر ایفا می‌کند. این توکن به صورت پارامتر قابل یادگیری تعریف شده و در طول فرآیند آموزش به روزرسانی می‌شود [۹، ۱۱].

۲.۹.۲ رمزگذار در مبدل‌های بینایی

رمزگذار در ترانسفورمرها همانند مبدل اصلی است [۵۹]، با این تفاوت که در مبدل‌های بینایی [۱۱] دیگر به رمزگشا نمی‌رویم. پس از عبور از بلاک‌های ترانسفورمر، در ساده‌ترین حالت یک لایه خطی^{۵۶} یا یک لایه MLP^{۵۷} بر روی بردار نهایی اعمال می‌شود و این لایه‌ها به تعداد کلاس‌ها خروجی می‌دهند. سپس خروجی هر لایه با گذر از تابع سافت‌مکس^{۵۸} به احتمال هر کلاس تبدیل می‌شود و در نهایت مدل کلاس با بیشترین احتمال را به عنوان خروجی پیش‌بینی می‌کند.



شکل ۱۰.۲: توکن توجه در مبدل‌های بینایی

در مبدل‌ها، هر لایه رمزگشا و رمزگذار با پردازش عمیق‌تر روی توالی ورودی، می‌تواند نمایش بهتری از ویژگی‌ها به دست بیاورد [۵۹]. تکرار چندین باره رمزگشا یا رمزگذار موجب می‌شود مدل بتواند ساختارهای پیچیده‌ای را یاد بگیرد و کیفیت و دقت آن در شناسایی توالی‌های طولانی و معانی

^{۵۶}Fully Connected

^{۵۷}Multi-Layer Perceptron

^{۵۸}softmax

پنهان افزایش یابد [۵۹، ۱۱]. در نتیجه، مدل با تعداد لایه‌های بیشتر اغلب عملکرد بهتری از خود نشان می‌دهد.

۱۰.۲ مبدل پنجره‌ای متحرک

ایده مبدل پنجره‌ای متحرک^{۵۹} از ترکیب چند مفهوم کلیدی در مدل‌های ترانسفورمر و شبکه‌های پیچشی شکل گرفت [۵۹، ۱۸، ۳۶].

یکی از بزرگترین مشکلات در ترانسفورمرهای اولیه، نیاز به محاسبات بسیار زیاد در زمانی بود که تصویر ورودی ابعاد بسیار بزرگی داشت [۱۱]. در ترانسفورمر معمولی هر پچ به تمامی پچ‌های دیگر توجه می‌کرد و در مواقعی که تعداد پچ‌ها زیاد می‌شد، هزینه محاسباتی و حافظه به شدت افزایش پیدا می‌کرد.

در شبکه‌های پیچشی، معماری معمولاً به صورت سلسله‌مراتبی پیش می‌رود [۱۸]؛ یعنی ابتدا ویژگی‌های محلی استخراج می‌شود، سپس با عمیق‌تر شدن لایه‌ها، این ویژگی‌ها در سطوح بالاتر با یکدیگر ترکیب می‌شوند. در مبدل پنجره‌ای متحرک [۳۶]، با دانش بر این موضوع توانسته‌اند هم هزینه‌های محاسباتی را کاهش دهند و هم دقت مدل را افزایش دهند.

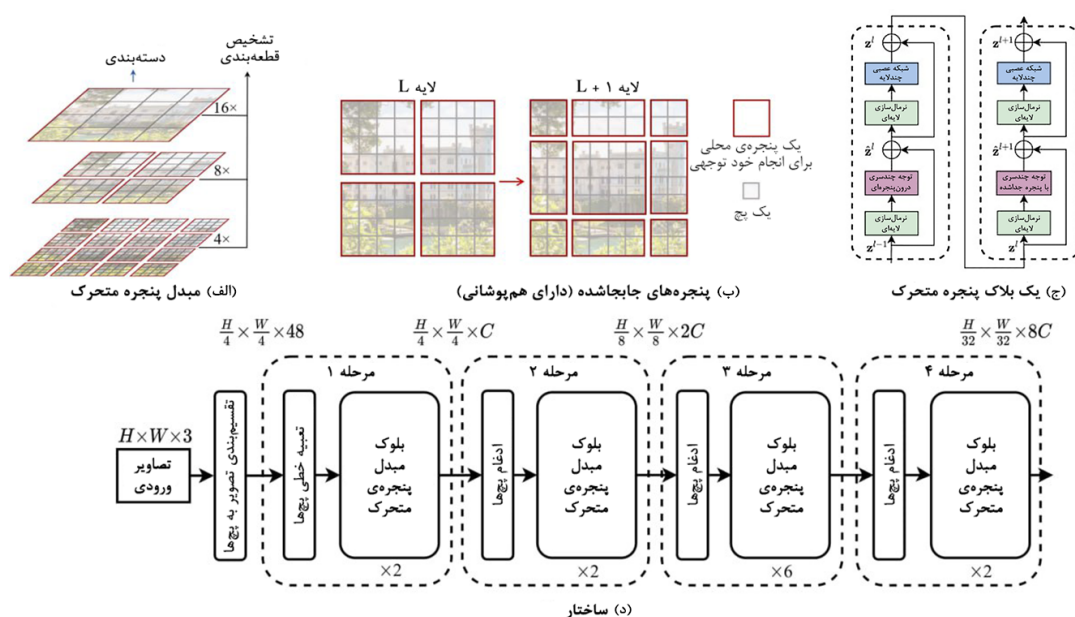
در مبدل پنجره‌ای متحرک، به جای آن‌که مدل به تمام پچ‌ها در یک سطح ویژگی نگاه کند، تصویر را به «پنجره‌های محلی»^{۶۰} تقسیم می‌کند و توجه را محدود به همان ناحیه می‌سازد [۳۶]. سپس با تکنیک جابه‌جایی^{۶۱} این پنجره‌ها در لایه‌های بعدی، توان مدل برای ترکیب اطلاعات از نواحی مختلف تصویر (و در نهایت دیدن کل تصویر) افزایش پیدا می‌کند. این رویکرد، ایده کلیدی‌ای بود که باعث شد مدل هم محاسبات سبک‌تری داشته باشد و هم بتواند ارتباط‌های جهانی^{۶۲} را در طول لایه‌ها به دست آورد.

^{۵۹} Swin Transformer

^{۶۰} Local Windows

^{۶۱} Shift

^{۶۲} Global



شکل ۱۱.۲: مبدل پنجره متحرک

یکی دیگر از ایده‌های مهم در در مبدل پنجره‌ای متحرک، کوچک کردن تدریجی نقشه ویژگی در طول معماری است؛ مشابه کاری که در ResNet یا شبکه‌های پیچشی انجام می‌شود [۱۸]. این امر ضمن کاهش هزینه محاسباتی، باعث می‌شود مدل بتواند با سطوح مختلفی از ویژگی‌ها کار کند و در نهایت خروجی نهایی با کیفیت‌تری ارائه دهد.

۱.۱۰.۲ قطعه‌بندی پیچ در مبدل پنجره متحرک

فرض کنیم تصویر ورودی I دارای ابعاد $(H \times W \times 3)$ باشد. گام نخست، تقسیم تصویر به پیچ‌های کوچک $(P \times P)$ است [۱۱]. اگر P اندازه پیچ ^{۶۳} باشد، آنگاه تعداد پیچ‌ها در بعد افقی و عمودی، به ترتیب $\frac{H}{P}$ و $\frac{W}{P}$ خواهد بود. هر پیچ را می‌توان به صورت یک بردار درآورد:

$$X_{\text{patch}} \in \mathbb{R}^{(P^2 \cdot 3)}.$$

^{۶۳} patch size

سپس کل تصویر به $\frac{H}{P} \times \frac{W}{P}$ پیچ تبدیل خواهد شد و در نتیجه، ماتریس X از کنار هم قرار گرفتن این پیچ‌ها به صورت زیر به دست می‌آید:

$$X \in \mathbb{R}^{\left(\frac{H}{P} \cdot \frac{W}{P}\right) \times (P^2 \cdot 3)}.$$

برخلاف مبدل بینایی اصلی این کار در مبدل پنجره متحرک با استفاده از کانولوشن^{۶۴} انجام می‌شود. در واقع سائز کرنل در کانولوشن همان فضای برداری هر پیچ هست که ما فرض می‌کنیم سائز کرنل برابر C است. پس در نتیجه

$$Z = X \cdot W_{\text{embed}} + b_{\text{embed}}, \quad Z \in \mathbb{R}^{\left(\frac{H}{P} \cdot \frac{W}{P}\right) \times C}. \quad (۱۴.۲)$$

در عمل، این عملیات معادل یک تبدیل خطی ساده است:

$$W_{\text{embed}} \in \mathbb{R}^{(P^2 \cdot 3) \times C}, \quad b_{\text{embed}} \in \mathbb{R}^C.$$

پس از این مرحله، ما در هر موقعیت (h, w) (از شبکه پیچ‌ها) یک بردار $z_{h,w} \in \mathbb{R}^C$ داریم. این ماتریس Z ورودی اولین مرحله (Stage) از مبدل‌های پنجره متحرک خواهد بود [۳۶].

هر بلوک مبدل پنجره متحرک از چند بخش اصلی تشکیل شده است [۳۶]. نخست، فرآیند پنجره‌بندی تصویر^{۶۵} یا در حالت دیگر پنجره‌بندی جابه‌جاشده^{۶۶} انجام می‌شود که ورودی را به پنجره‌های کوچک تقسیم می‌کند. سپس، در هر پنجره مکانیزم توجه چندسر پنجره‌ای^{۶۷} اعمال می‌گردد تا روابط درون هر پنجره به‌طور موازی مدل شود. پس از آن، خروجی‌ها از طریق اتصالات میان‌بری^{۶۸} و نرمال‌سازی لایه‌ای^{۶۹} پایدار می‌شوند. در نهایت، داده‌ها وارد مسیر پرسپترون چندلایه

^{۶۴}convolution

^{۶۵}Window Partition

^{۶۶}Shifted Window Partition

^{۶۷}Window Multi-Head Self Attention

^{۶۸}Skip Connection

^{۶۹}Layer Norm

^{۷۰} می‌شوند که معمولاً شامل دو لایه تمام‌متصل همراه با توابع فعال‌سازی غیرخطی (مانند GeLU) و همچنین اتصالات میان‌بری و نرمال‌سازی لایه‌ای است. این اجزا در کنار هم موجب می‌شوند که بلوک مبدل پنجره متحرک بتواند وابستگی‌های محلی را با کارایی بالا مدل کرده و در عین حال قابلیت تعمیم‌پذیری مناسبی در لایه‌های عمیق‌تر داشته باشد.

۲.۱۰.۲ توجه چندسر پنجره‌ای

تعریف پنجره‌های محلی

در مبدل‌های پنجره متحرک، به جای آنکه تمام پیکسل‌های یک نقشه ویژگی بزرگ به‌طور هم‌زمان در محاسبات توجه دخالت داده شوند، نقشه ویژگی به قطعه‌های کوچکی با اندازه $(M \times M)$ تقسیم می‌شود. این قطعه‌های کوچک را «پنجره‌های محلی» می‌نامند. اگر اندازه نقشه ویژگی در یک لایه $(H' \times W')$ باشد، پس از تقسیم آن به پنجره‌های $(M \times M)$ ، تقریباً $\frac{H'}{M}$ پنجره در راستای طول و $\frac{W'}{M}$ پنجره در راستای عرض خواهیم داشت. برای سادگی معمولاً فرض می‌شود که H' و W' مضربی از M باشند تا تقسیم بدون باقی‌مانده انجام گیرد. هر پنجره محلی شامل M^2 موقعیت مکانی است و در هر موقعیت یک بردار ویژگی با بعد C قرار دارد. به بیان ساده، می‌توان نقشه ویژگی را مانند یک صفحه بزرگ شطرنج تصور کرد که به مربع‌های کوچک $(M \times M)$ تقسیم می‌شود؛ سپس در هر مربع تنها محاسبات توجه همان بخش انجام می‌گیرد. این شیوه باعث می‌شود تعداد پیکسل‌هایی که درگیر محاسبه توجه هستند به‌طور قابل توجهی کاهش یابد و در نتیجه هزینه محاسباتی نیز کمتر شود.

۳.۱۰.۲ توجه

برای هر بلوک، ابتدا بردارهای پرسش، کلید، مقدار ساخته می‌شوند. اگر $z_i \in \mathbb{R}^C$ بردار ورودی مربوط به موقعیت i باشد، آنگاه:

$$q_i = z_i W_Q, \quad k_i = z_i W_K, \quad v_i = z_i W_V, \quad (۱۵.۲)$$

که

$$W_Q, W_K, W_V \in \mathbb{R}^{C \times d}.$$

پارامتر d معمولاً به صورت $\frac{C}{h}$ در نظر گرفته می شود که در آن h تعداد سربندی سرها است. در توجه چندسر، خروجی نهایی با ترکیب h سر توجه محاسبه می شود. در یک سر توجه، توجه به صورت زیر تعریف می شود:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d}}\right) V, \quad (۱۶.۲)$$

که در آن:

• Q, K, V به ترتیب ماتریس هایی هستند که از کنار هم قرار دادن q_i, k_i, v_i (برای تمام پیکسل های آن پنجره) ساخته می شوند.

• $\sqrt{d}$: عامل مقیاس کننده برای جلوگیری از بزرگ شدن بیش از حد ضرب داخلی است.

در مبدل های پنجره متحرک، این محاسبات به صورت پنجره ای انجام می شوند؛ یعنی برای هر پنجره، تنها پیکسل های داخل همان پنجره در ماتریس های Q, K و V لحاظ می شوند. به این ترتیب، زمان محاسبه و مصرف حافظه به شدت کاهش می یابد (در مقایسه با مبدل های بینایی که همه چیز را با هم مقایسه می کند).

تعداد سربندی h معمولاً طوری انتخاب می شود که $C = h \times d$ خروجی هر سر پس از محاسبه توجه به صورت زیر با هم ادغام می شوند:

$$\text{MultiHead}(Q, K, V) = [\text{head}_1, \text{head}_2, \dots, \text{head}_h] W_O, \quad (17.2)$$

که

$$\text{head}_j = \text{Attention}(Q_j, K_j, V_j), \quad W_O \in \mathbb{R}^{C \times C}$$

ماتریس ترکیب نهایی است.

۴.۱۰.۲ پنجره متحرک جابه‌جاشده

در مبدل‌های پنجره متحرک، ایده «پنجره‌های جابه‌جاشده»^{۷۱} به منظور یادگیری ارتباط میان پیکسل‌های واقع در پنجره‌های مجاور معرفی شده است [۳۶]. اگر تنها از پنجره‌های ثابت (بدون جابه‌جایی) استفاده شود، هر بلوک تصویری صرفاً با پیکسل‌های همان پنجره در ارتباط خواهد بود و تبادل اطلاعات با نواحی مرزی و مجاور به‌خوبی صورت نخواهد گرفت. برای رفع این محدودیت، مبدل پنجره متحرک از یک تکنیک ساده اما مؤثر بهره می‌برد: در یک لایه، محاسبات توجه روی پنجره‌های محلی ثابت انجام می‌شود، سپس در لایه بعدی همان پنجره‌ها به میزان مشخصی در راستای افقی و عمودی جابه‌جا می‌شوند. این جابه‌جایی باعث می‌شود نواحی مرزی نیز در محاسبات توجه دخیل شوند و در نتیجه پیکسل‌ها بتوانند در پنجره‌های مختلف حضور یابند. بدین ترتیب، تبادل اطلاعات میان بخش‌های مجاور تصویر به‌طور کامل‌تری انجام می‌گیرد و مدل قادر خواهد بود وابستگی‌های گسترده‌تری را در تصویر بیاموزد.

توجه چندسری پنجره‌ای

در توجه چندسری پنجره‌ای^{۷۲}، نقشه ویژگی به پنجره‌های $(M \times M)$ تقسیم می‌شود [۳۶]. هیچ جابه‌جایی در این تقسیم‌بندی وجود ندارد؛ یعنی اگر نقشه ویژگی را یک مستطیل بزرگ در نظر

Shifted Windows^{۷۱}
W-MSA^{۷۲}

بگیریم، آن را شبیه کاشی‌کاری یا شطرنج‌بندی به بلوک‌های مربعی ($M \times M$) برش می‌زنیم. در این حالت، پیکسل‌های هر پنجره فقط با همدیگر (درون همان پنجره) ارتباط برقرار می‌کنند.

توجه چندسری پنجره‌ای جابه‌جاشده

مطابق شکل ۱۲.۲، بعد از اینکه بلوک اول (توجه چندسری پنجره‌ای) کارش تمام شد، در بلوک دوم، قبل از تقسیم‌بندی به پنجره‌های ($M \times M$)، نقشه ویژگی را جابه‌جا می‌کنیم [۳۶]. در مقاله اصلی، این مقدار جابه‌جایی معمولاً نیم اندازه پنجره $\frac{M}{2}$ در راستای افقی و عمودی است. به این ترتیب:

پیکسل‌هایی که پیش از این در دو پنجره جداگانه قرار داشتند، ممکن است حالا به دلیل جابه‌جایی وارد یک پنجره مشترک شوند. مدل حالا می‌تواند بین این پیکسل‌های «مرزی» نیز توجه برقرار کند و اطلاعات را بهتر مبادله کند.

با این جابه‌جایی، بخشی از پیکسل‌ها در نقشه ویژگی از یک طرف «خارج» می‌شوند. برای اینکه این پیکسل‌ها را از دست ندهیم، از ترفندی به نام جابه‌جایی چرخه‌ای^{۷۳} استفاده می‌شود. در جابه‌جایی چرخه‌ای، پیکسل‌هایی که از سمت راست بیرون می‌روند دوباره از سمت چپ وارد می‌شوند و بالعکس؛ درست شبیه وقتی که یک تصویر را به صورت حلقه‌ای اسکرول می‌کنیم^{۷۴}. مثالی از جابه‌جایی چرخه‌ای در شکل ۱۲.۲ آمده است.

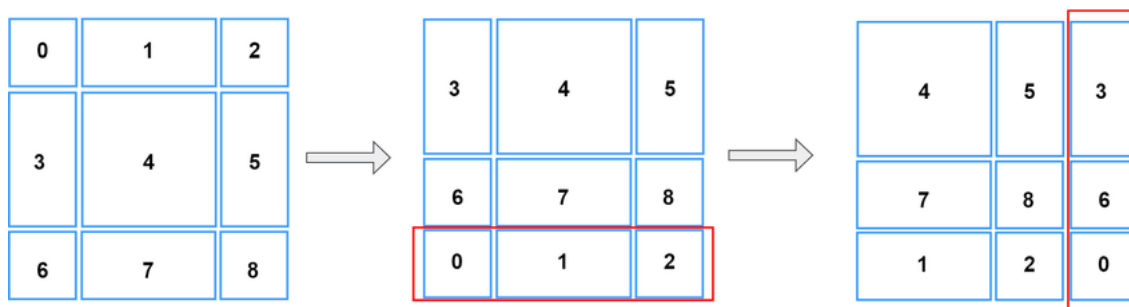
در بلوک اول (بدون جابه‌جایی)، پنجره‌ها ثابت‌اند و پیکسل‌های مرزی در هر پنجره ممکن است فرصت کافی برای تبادل اطلاعات با پیکسل‌های مرزی پنجره کناری را نداشته باشند.

در بلوک دوم (جابه‌جاشده)، مرزهای پنجره‌ها تغییر می‌کند و برخی پیکسل‌هایی که قبلاً در پنجره‌های جدا بودند، اکنون در یک پنجره مشترک‌اند؛ در نتیجه مدل می‌تواند رابطه و همبستگی بین آن‌ها را هم یاد بگیرد.

این جابه‌جایی و قرارگیری مجدد پیکسل‌ها کنار هم در نهایت کمک می‌کند تا مدل بتواند

Cyclic Shift^{۷۳}

Wrap around^{۷۴}



شکل ۱۲.۲: جابه‌جایی چرخه‌ای

اطلاعات کل تصویر را با هزینه محاسباتی کمتر (نسبت به توجه سراسری کامل) در اختیار داشته باشد [۳۶].

اگر بخواهیم با مثال توضیح دهیم، فرض کنید در یک تابلوی شطرنجی، خانه‌های کناری همدیگر را «نمی‌بینند» چون در دو بلوک مختلف هستند. اما اگر کمی تابلوی شطرنجی را به سمت بالا-چپ یا پایین-راست جابه‌جا کنیم، حالا بخشی از آن خانه‌ها وارد یک بلوک واحد می‌شوند و اطلاعاتشان با هم ترکیب می‌شود. سپس به‌طور دوره‌ای، گوشه‌های اضافی را به آن سمت دیگر تابلوی شطرنجی می‌آوریم تا هیچ چیز از دست نرود.

به این شکل، سری اول و دوم بلوک‌های مبدل‌های پنجره متحرک تکمیل‌کننده یکدیگر می‌شوند [۳۶]:

- بلوک اول: محاسبه توجه در چهارچوب پنجره‌های ثابت.
- بلوک دوم: محاسبه توجه در پنجره‌های جابه‌جاشده که منجر به تعامل بیشتر بین مرزهای مختلف می‌شود.

۵.۱۰.۲ پرسپترون چندلایه

پس از انجام توجه چندسری پنجره‌ای جابه‌جاشده خروجی به یک مسیر MLP می‌رود [۳۶]. ساختار این MLP به‌صورت زیر است:

$$X' = \text{GELU}(XW_1 + b_1) W_2 + b_2, \quad (18.2)$$

که در آن

$$W_1 \in \mathbb{R}^{C \times (rC)}, \quad W_2 \in \mathbb{R}^{(rC) \times C}$$

هستند و r معمولاً ضریب افزایش بعد را نشان می‌دهد (مثلاً ۴).

تابع فعال‌ساز GELU (یا ReLU و سایر توابع) نیز در این جا قابل استفاده است [۱۹].

۶.۱۰.۲ ترکیب پیچ‌ها

در مدل مبدل‌های پنجره متحرک، ساختار سلسله‌مراتبی به این معناست که ما در چند مرحله (Stage) مختلف، نقشه ویژگی را کوچک‌تر می‌کنیم و در عین حال، عمق (تعداد کانال‌های ویژگی) را افزایش می‌دهیم. هدف اصلی از این کار عبارت است از:

- استخراج ویژگی‌های سطح بالاتر: وقتی نقشه ویژگی کوچک‌تر می‌شود، هر واحد از نقشه ویژگی بیانگر بخش گسترده‌تری از تصویر اصلی است؛ پس مدل به تدریج جزئیات محلی را با درک کلی‌تری از تصویر جایگزین می‌کند [۱۸].

- کاهش هزینه محاسبات: در مراحل بعدی، چون ابعاد فضایی کمتر می‌شود، مدل راحت‌تر می‌تواند با ویژگی‌های جدید کار کند (چون مثلاً به جای $(H \times W)$ پیکسل، تعداد کمتری پیکسل داریم) [۳۶].

پس از چندین بلوک پردازشی، نقشه ویژگی، ابعادی به شکل $(\frac{H}{P}, \frac{W}{P})$ با تعداد کانال C دارد. این یعنی پس از برش دادن تصویر به پیچ‌ها و گذر از چند لایه، اکنون یک نقشه ویژگی داریم که کوچک‌تر از تصویر اصلی است، اما هنوز ممکن است خیلی بزرگ باشد.

در مرحله بعد (Stage بعدی)، می‌خواهیم این نقشه را نصف کنیم (یعنی طول و عرض را دو برابر کوچک کنیم) و در عوض عمق کانال را دو برابر کنیم (تا ظرفیت مدل در استخراج ویژگی‌های پیچیده‌تر بیشتر شود). برای انجام این کار از فرایندی به نام ترکیب پچ‌ها استفاده می‌کنیم [۳۶]:

۱. انتخاب بلوک‌های (2×2)

ابتدا نقشه ویژگی را در بُعد مکانی به بلوک‌های (2×2) تقسیم می‌کنیم. اگر $Z_{i,j}$ ویژگی مکان (i, j) باشد، یک بلوک (2×2) شامل چهار پیکسل است:

$$Z_{2i,2j}, \quad Z_{2i,2j+1}, \quad Z_{2i+1,2j}, \quad Z_{2i+1,2j+1}.$$

۲. ادغام ویژگی‌های چهار پیکسل

برای هر بلوک (2×2) ، این چهار پیکسل را در بُعد کانال به هم می‌چسبانیم. اگر هر پیکسل یک بردار از بعد C باشد، اکنون بعد حاصل از کنار هم گذاشتن این چهار پیکسل می‌شود $4C$. نام این بردار ادغام‌شده را Z' می‌گذاریم.

۳. لایه خطی برای تغییر بعد

وقتی چهار بردار C - بعدی را کنار هم می‌گذاریم، یک بردار $4C$ - بعدی شکل می‌گیرد. حال با یک لایه خطی، بعد $4C$ را به بعد جدیدی تبدیل می‌کنیم. معمولاً این بعد جدید برابر $2C$ در نظر گرفته می‌شود؛ یعنی دو برابر بزرگ‌تر از قبل اما نه چهار برابر:

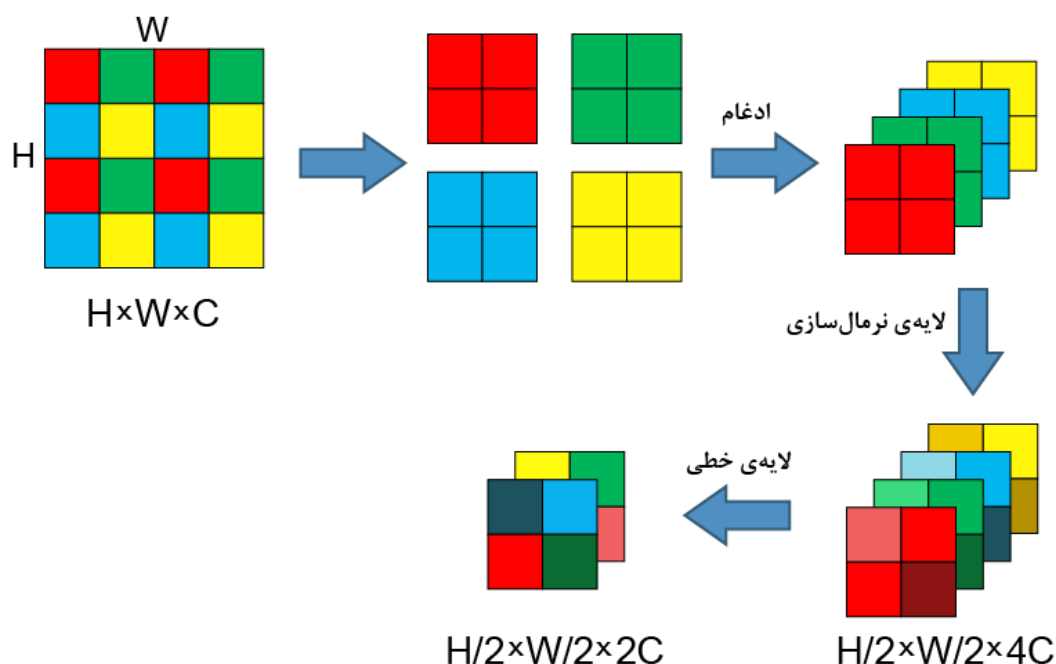
$$Z' \mapsto Z'' = Z' W_{\text{merge}} + b_{\text{merge}}, \quad (19.2)$$

که بعد ویژگی را از $4C$ به $2C$ کاهش می‌دهد.

۴. کاهش ابعاد مکانی

در عین حال، وقتی هر چهار پیکسل (2×2) را ادغام می‌کنیم، نقشه ویژگی ما ابعاد فضایی $(\frac{H}{2P} \times \frac{W}{2P})$ خواهد داشت (چون هر بلوک (2×2) تبدیل به یک بردار می‌شود).

به عبارت دیگر، تعداد نقاط مکانی نصف می‌شود (هم در طول و هم در عرض)، اما کانال از C به $2C$ افزایش می‌یابد.



شکل ۱۳.۲: ادغام پیچ‌ها

در شبکه‌های کانولوشنی، مرتبا از لایه‌های ادغام^{۷۵} یا کانولوشن با گام^{۷۶} برای کوچک کردن ابعاد استفاده می‌شود تا اطلاعات سطح بالاتر (مثل ساختار کلی اشیا) راحت تر استخراج شود [۱۸]. در مبدل پنجره متحرک هم همین ایده سلسله مراتب را به دنیای مبدل‌ها آورده است [۳۶]. همچنین اگر ابعاد فضایی را کم نکنیم، هزینه توجه به شدت زیاد می‌شود (چون باید در هر لایه برای همه پیکسل‌ها توجه محاسبه گردد).

در معماری کلی مبدل‌های پنجره متحرک، پس از Stage 1 و عبور از بلوک‌های توجه چندسر پنجره‌ای و توجه چندسر پنجره‌ای جابه‌جاشده، عملیات ادغام پیچ‌ها انجام می‌شود. سپس در Stage

^{۷۵} Pooling
^{۷۶} Stride-Convolution

2، ویژگی‌های کوچک‌تری داریم، اما تعداد کانال‌ها افزایش یافته است [۳۶]. مشابه معماری‌های کانولوشنی، با افزایش عمق^{۷۷}، ابعاد فضایی کاهش و تعداد کانال‌ها افزایش پیدا می‌کند. در انتهای Stage آخر، خروجی به یک لایه FC داده می‌شود تا تعداد کلاس‌ها را پیش‌بینی کند. پس از گذر از Softmax، احتمال هر کلاس به‌دست می‌آید و مدل در نهایت کلاس نهایی را برمی‌گزیند.

فصل ۳

روش‌های پیشنهادی

استفاده از روش‌های تانسوری در شبکه‌های عصبی چندلایه

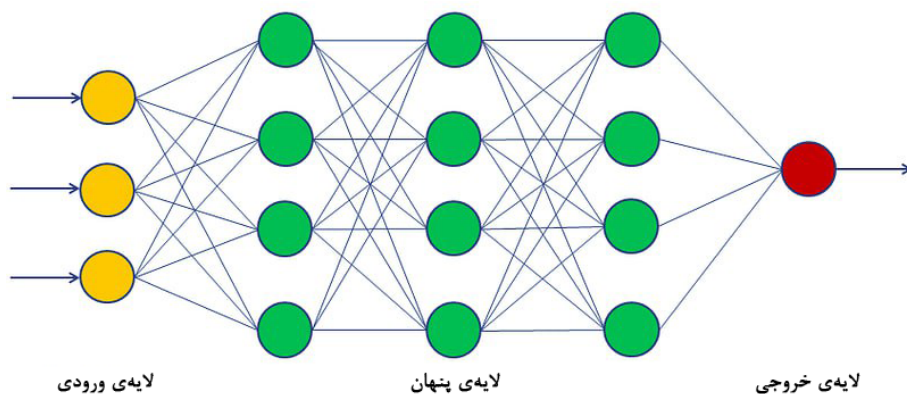
در سال‌های اخیر، استفاده از روش‌های تانسوری به‌عنوان روشی نوین در بهینه‌سازی معماری‌های شبکه‌های عصبی، به‌ویژه در مدل‌هایی که تعداد پارامترهای آن‌ها بسیار زیاد است، مانند شبکه‌های عصبی چندلایه^۱، توجه بسیاری را به خود جلب کرده است [۴۸]. تانسورها تعمیمی از ماتریس‌ها به ابعاد بالاتر هستند و به‌طور طبیعی برای نمایش داده‌های چندبعدی همچون تصاویر، ویدیوها یا سری‌های زمانی چندکاناله مناسب‌اند. بهره‌گیری از ساختار تانسوری در معماری شبکه، این امکان را فراهم می‌سازد که بدون نیاز به فشردن سازی اولیه (مانند مسطح‌سازی^۲ ورودی)، اطلاعات ساختاری میان ابعاد مختلف حفظ شده و مدل بتواند از روابط درون‌تعاملی موجود میان این ابعاد بهره‌برداری نماید [۴۸].

در معماری سنتی شبکه عصبی چندلایه، نگاشت از ورودی $\mathbf{x} \in \mathbb{R}^n$ به خروجی $\mathbf{y} \in \mathbb{R}^m$ به‌وسیله ضرب ماتریسی انجام می‌گیرد:

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$$

MLP^۱
Flatten^۲

که در آن $W \in \mathbb{R}^{m \times n}$ ماتریس وزن و b بردار بایاس است.



شکل ۱.۳: شبکه عصبی پرسپترون چندلایه

در مقابل، در روش‌های تانسوری، وزن‌ها به صورت یک تانسور مرتبه بالاتر مدل‌سازی می‌شوند و نداشت ورودی به خروجی با استفاده از ضرب‌های چندحالتی (ضرب تانسوری در ابعاد مختلف) صورت می‌پذیرد:

$$y = \mathcal{W} \times_1 \mathbf{x}_1 \times_2 \mathbf{x}_2 \times_3 \cdots + \mathbf{b} \quad (1.3)$$

در این رابطه، $\mathcal{W}$ یک تانسور وزن است و عملگر $\times_n$ نشان‌دهنده ضرب تانسوری در بعد n ام می‌باشد.

روش‌هایی نظیر TCL^3 و TRL^4 به عنوان نمونه‌هایی از این رویکرد، با بهره‌گیری از تکنیک‌های تجزیه تانسوری همچون تاکر^۵ یا تجزیه CP، نه تنها باعث کاهش چشمگیر در تعداد پارامترها می‌شوند، بلکه ساختار چندبعدی داده‌ها را نیز حفظ می‌نمایند [۲۹، ۳۰].

^۳Tensor Contraction Layer
^۴Tensor Regression Layer
^۵Tucker

مزایای استفاده از روش‌های تانسوری

استفاده از روش‌های تانسوری در شبکه‌های عصبی چندلایه مزایای متعددی به همراه دارد که برخی از مهم‌ترین آن‌ها عبارت‌اند از:

- کاهش چشمگیر تعداد پارامترها: با بهره‌گیری از فشرده‌سازی تانسوری، می‌توان ابعاد تانسور وزن‌ها را به گونه‌ای کاهش داد که بدون افت محسوس در عملکرد مدل، مصرف حافظه و پیچیدگی محاسباتی به‌طور قابل توجهی کاهش یابد [۳۳، ۳۰].
- حفظ ساختار داده‌های ورودی: برخلاف روش‌های سنتی که در آن‌ها داده‌ها پیش از ورود به لایه‌های چگال^۶ باید مسطح‌سازی^۷ شوند، استفاده از ساختار تانسوری این امکان را فراهم می‌کند که ساختار فضایی، زمانی یا کانالی داده‌ها حفظ شده و ارتباط میان ابعاد مختلف ورودی بهتر درک و پردازش شود [۳۰].

محدودیت‌ها و چالش‌ها

با وجود مزایای متعدد، بهره‌گیری از روش‌های تانسوری در معماری‌های شبکه‌های عصبی با چالش‌ها و محدودیت‌هایی نیز همراه است که در ادامه به برخی از مهم‌ترین آن‌ها اشاره می‌شود:

- پیچیدگی بالاتر در پیاده‌سازی: پیاده‌سازی لایه‌های مبتنی بر عملیات تانسوری معمولاً به ابزارها و کتابخانه‌های خاصی همچون Tensorly یا Tensor Toolbox نیاز دارد. این موضوع فرآیند طراحی و توسعه مدل را پیچیده‌تر از استفاده از لایه‌های استاندارد مانند شبکه عصبی چندلایه می‌سازد [۲۹].
- بهینه‌سازی دشوارتر: فرآیند آموزش مدل‌های تانسوری می‌تواند نسبت به مدل‌های معمولی کندتر باشد. الگوریتم‌های مبتنی بر گرادیان ممکن است در فضای پارامتری تانسورها با

Dense^۶
Flatten^۷

سطوح خطای غیرهموار یا چندوجهی مواجه شوند که روند همگرایی را دشوار می‌کند [۶۰].

- احتمال کاهش دقت در فشرده‌سازی شدید: در صورتی که میزان فشرده‌سازی تانسورها بیش از حد بالا باشد، مدل ممکن است توانایی لازم برای نمایش روابط غیرخطی و الگوهای پیچیده را از دست داده و در نتیجه، دقت نهایی پیش‌بینی کاهش یابد [۳۰].

۱.۰.۳ تجزیه تاکر

یکی از مهم‌ترین روش‌های تجزیه تانسوری، تجزیه تاکر است که برای نخستین بار توسط تاکر معرفی شد [۵۷]. این روش نوعی تجزیه با رتبه پایین است که هدف آن نمایش یک تانسور مرتبه بالا به وسیله یک هسته (core) با ابعاد کوچک‌تر و مجموعه‌ای از ماتریس‌های فاکتور در هر بُعد است. برخلاف تجزیه CP که تنها بر پایه جمع ترکیبی از بردارها استوار است، در تجزیه تاکر امکان مدل‌سازی وابستگی‌های چندبعدی پیچیده‌تر نیز فراهم می‌شود [۲۸].

فرض کنید یک تانسور ورودی به شکل زیر داشته باشیم:

$$\mathcal{X} \in \mathbb{R}^{I_1 \times I_2 \times \dots \times I_N},$$

تجزیه تاکر آن را می‌توان به صورت زیر نوشت:

$$\mathcal{X} \approx \mathcal{G} \times_1 U^{(1)} \times_2 U^{(2)} \dots \times_N U^{(N)},$$

که در آن:

- $\mathcal{G} \in \mathbb{R}^{R_1 \times R_2 \times \dots \times R_N}$ تانسور هسته با ابعاد فشرده‌تر است،

- $U^{(n)} \in \mathbb{R}^{I_n \times R_n}$ ماتریس‌های فاکتور در بُعد n ام هستند،

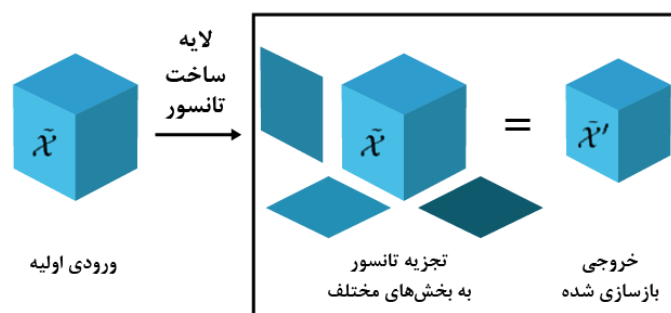
- عملگر $\times_n$ ضرب مد- n را نشان می‌دهد.

تجزیه تاکر از این جهت اهمیت دارد که می‌تواند تعداد پارامترهای موردنیاز برای نمایش یک تانسور بزرگ را به‌طور قابل توجهی کاهش دهد و در عین حال ساختار چندبعدی داده‌ها را حفظ کند. این ویژگی باعث شده که تاکر به‌عنوان یکی از پرکاربردترین روش‌ها در فشرده‌سازی و مدل‌سازی شبکه‌های عصبی مورد استفاده قرار گیرد [۴۸، ۳۱].

۲.۰.۳ لایه فشرده‌سازی تانسوری

در بسیاری از مدل‌های یادگیری عمیق — به‌ویژه شبکه‌های عصبی پیچشی^۸ فعال‌سازی‌های لایه‌های میانی به‌شکل تانسورهایی با مرتبه بالا ظاهر می‌شوند. به‌طور سنتی، برای اعمال لایه‌های کاملاً متصل^۹، این تانسورها ابتدا با عملیات مسطح‌سازی به بردار تبدیل و سپس به فضای خروجی نگاشت می‌شوند؛ روشی که ساختار چندخطی^{۱۰} داده را از بین برده و به‌تبع، تعداد پارامترها به‌صورت افسارگسیخته افزایش می‌یابد [۳۰].

در پاسخ به این چالش، لایه‌ای با عنوان لایه فشرده‌سازی تانسوری یا به اختصار TCL معرفی شده است. این لایه بدون نیاز به مسطح‌سازی، هر مد (mode) از تانسور ورودی را مستقل فشرده‌سازی می‌کند و ساختار چندخطی داده را حفظ می‌نماید [۲۹، ۳۰].



شکل ۲.۳: لایه فشرده‌سازی تانسوری

^۸ Convolution
^۹ Fully Connected
^{۱۰} multilinear structure

فرمول‌بندی ریاضی

فرض کنید تانسور فعال‌سازی ورودی:

$$\mathcal{X} \in \mathbb{R}^{S \times I_0 \times I_1 \times \dots \times I_N}$$

باشد (که S اندازه دسته‌بندی و I_k ها ابعاد کانال/فضایی را نشان می‌دهند). هدف لایه TCL ، نگاشت:

$$\mathcal{X}' = \mathcal{X} \times_1 V^{(0)} \times_2 V^{(1)} \dots \times_{N+1} V^{(N)} \quad (2.3)$$

است، که در آن:

$$V^{(k)} \in \mathbb{R}^{R_k \times I_k}, \quad \forall k = 0, \dots, N.$$

با این عملگرهای ضرب چندحالتی ($\times_n$) تعداد پارامترها به‌شکلی چشمگیر کاهش می‌یابند [۲۹].

تحلیل تعداد پارامترها

در TCL ، کل پارامترهای قابل آموزش برابر است با:

$$TCL \text{ پارامترهای} = \sum_{k=0}^N I_k \cdot R_k \quad (3.3)$$

در حالی‌که در لایه متصل سنتی (پس از مسطح‌سازی):

$$FC \text{ پارامترهای} = \left(\prod_{k=0}^N I_k \right) \cdot O \quad (4.3)$$

که O تعداد نورون‌های خروجی لایه است. نمایش ریاضی بالا به‌وضوح نشان می‌دهد که TCL با جایگزینی ضرب با جمع ساده، به‌صرفه‌سازی قابل توجهی در حافظه و محاسبات مدل منجر می‌شود

[۲۹، ۳۰].

۳.۰.۳ لایه رگرسیون تانسوری (TRL)

این لایه، خروجی مدل را مستقیماً از تانسور ورودی تولید می‌کند، بدون مسطح‌سازی؛ به‌طوری‌که ساختار چندبعدی ورودی حفظ و نگاشت خروجی نیز به‌صورت مرتبه پایین^{۱۱} مدل شده است [۳۰].

فرمول‌بندی ریاضی

فرض مجدد تانسور:

$$\mathcal{X} \in \mathbb{R}^{S \times I_0 \times I_1 \times \dots \times I_N}$$

و تانسور وزن:

$$\mathcal{W} \in \mathbb{R}^{I_0 \times I_1 \times \dots \times I_N \times O}.$$

نگاشت خروجی:

$$\mathbf{Y} = \langle \mathcal{X}, \mathcal{W} \rangle_N + \mathbf{b} \quad (5.3)$$

در اینجا عملگر $\langle \cdot, \cdot \rangle_N$ ضرب داخلی روی N بعد اول $\mathcal{W}$ و N بعد اول $\mathcal{X}$ را نشان می‌دهد [۳۰].

برای کاهش پارامترها، $\mathcal{W}$ با کمک تجزیه تاکر به شکل زیر نوشته می‌شود:

$$\mathcal{W} = \mathcal{G} \times_0 U^{(0)} \times_1 U^{(1)} \dots \times_N U^{(N)} \times_{N+1} U^{(N+1)}, \quad (6.3)$$

که $\mathcal{G}$ هسته کم‌مرتبه^{۱۲} و $U^{(k)}$ ماتریس‌های فشرده‌سازی هستند [۳۰].

فرمول نهایی:

$$\mathbf{Y} = \langle \mathcal{X} \times_0 (U^{(0)})^\top \dots \times_N (U^{(N)})^\top, \mathcal{G} \times_{N+1} U^{(N+1)} \rangle_N + \mathbf{b} \quad (7.3)$$

Low-Rank^{۱۱}

Core Tensor^{۱۲}

که به صورت محاسباتی بهینه‌تر عمل می‌کند [۳۰].

تحلیل تعداد پارامترها

در لایه رگرسیون تانسوری با تجزیه تاکر^{۱۳}، تعداد پارامترها برابر است با:

$$\text{TRL پارامترهای} = \left(\prod_{k=0}^{N+1} R_k \right) + \left(\sum_{k=0}^N I_k R_k \right) + R_{N+1} \cdot O, \quad (۸.۳)$$

که معمولاً بسیار کمتر از تعداد پارامترهای لایه FC است، به‌ویژه هنگامی که $R_k \ll I_k$ انتخاب شوند [۳۰].

۴.۰.۳ چرا در مبدل‌های بینایی از تانسور استفاده می‌کنیم؟

۱. کاهش تعداد پارامترها و حافظه مصرفی

یکی از چالش‌های اصلی در معماری‌های مبتنی بر مبدل‌های بینایی، رشد نمایی تعداد پارامترها در لایه‌های کاملاً متصل، به‌ویژه در بخش‌های انتهایی شبکه است. با جایگزینی این لایه‌ها با ساختارهای تانسوری مانند TCL و TRL، می‌توان از ساختار چندبعدی داده بهره برده و از طریق تجزیه‌های کم‌مرتبه، تعداد پارامترها را به صورت چشمگیری کاهش داد [۴۸، ۲۹، ۳۰]. این جایگزینی نه تنها موجب صرفه‌جویی در حافظه می‌شود، بلکه پیچیدگی محاسباتی مدل را نیز کاهش داده و امکان به‌کارگیری آن را در محیط‌های کم‌منبع (مانند دستگاه‌های لبه‌ای و موبایل) فراهم می‌سازد [۱۷].

۲. افزایش تفسیرپذیری با حفظ ساختار چندخطی داده

در حالی که عملیات مسطح‌سازی بر روی تانسورهای ورودی، ساختار ارتباطی میان ابعاد داده را از بین می‌برد، استفاده از لایه‌های تانسوری این ساختار چندخطی را حفظ می‌کند. این امر نه تنها مدل را برای کار با داده‌های فضایی یا زمانی چندکاناله مطلوب‌تر می‌سازد بلکه قابلیت تفسیرپذیری پیش‌بینی‌های مدل را نیز ارتقا می‌دهد [۱۷].

^{۱۳}Tucker

۳. کاهش نیاز به داده‌های آموزشی بزرگ

مدل‌های بینایی مبتنی بر مبدل، به دلیل فقدان سوگیری‌های مکانی ذاتی شبکه‌های پیچشی، نیازمند حجم عظیمی از داده برای آموزش مؤثر هستند. با استفاده از لایه‌های تانسوری مانند TRL و TCL که نداشت‌ها را به صورت چندخطی و فشرده مدل‌سازی می‌کنند و ساختار درونی داده را حفظ می‌کنند، می‌توان از ظرفیت مدل به صورت بهینه‌تر بهره برد. این ساختار نه تنها کمک می‌کند وابستگی‌های میان‌بُعدی بهتر درک شوند بلکه از بیش‌برازش در شرایط داده محدود جلوگیری می‌کند؛ به عنوان مثال، در روش FacT^{۱۴}، فقط 0.01% پارامترهای مبدل‌ها برای تعداد بسیار کمی از نمونه‌های آموزشی آموزش داده می‌شود و با صرفه‌جویی قابل توجه در پارامترها، عملکردی مشابه یا بهتر از تنظیم کامل^{۱۵} ارائه می‌دهد [۲۵].

۵.۰.۳ روش تانسوری مبدل پنجره متحرک:

۶.۰.۳ پیاده‌سازی مرحله تعبیه پیچ با استفاده از فشرده‌سازی تانسوری

در معماری‌های کلاسیک مبتنی بر مبدل‌های بینایی^{۱۶} و همچنین در ساختار مبدل‌های پنجره‌ای^{۱۷}، مرحله اولیه پردازش تصویر شامل تقسیم تصویر به قطعات کوچک (پیچ‌ها) و سپس نگاشت هر پیچ به یک بردار نهفته با ابعاد ثابت است. این نگاشت معمولاً با استفاده از یک لایه خطی^{۱۸} یا پیچشی^{۱۹} با کرنل و گام برابر با اندازه پیچ انجام می‌شود.

در روش تانسوری به جای استفاده از نگاشت برداری ساده، از لایه‌ی فشرده‌سازی تانسوری^{۲۰} برای تبدیل هر پیچ به یک تانسور چندبُعدی در فضای ویژگی استفاده شده است. این نگاشت تانسوری نه تنها باعث حفظ ساختار چندخطی پیچ‌ها می‌شود، بلکه با فشرده‌سازی مؤثر، منجر به

^{۱۴} Factor-Tuning^{۱۵} full-tuning^{۱۶} Vision Transformer^{۱۷} Swin Transformer^{۱۸} Linear^{۱۹} Convolution^{۲۰} Tensor Contraction Layer

کاهش پارامترها می‌شود.

در ابتدا فرض می‌کنیم ورودی مدل تصویری با اندازه‌ی (B, C, H, W) باشد که در آن:

• B اندازه‌ی دسته‌ی آموزشی^{۲۱} است.

• C تعداد کانال‌های تصویر (برای مثال ۳ در تصاویر رنگی).

• $H \times W$ ابعاد تصویر است.

این تصویر با استفاده از پچ‌هایی با اندازه‌ی $(P \times P)$ به $\frac{H}{P} \times \frac{W}{P}$ قطعه تقسیم می‌شود. سپس با استفاده از عملیات بازآرایی^{۲۲}، ساختار ورودی به شکل زیر تبدیل می‌شود:

$$\mathcal{X} \in \mathbb{R}^{B \times P_1 \times P_2 \times C_1 \times C_2 \times C_3}$$

که در آن:

• $P_1 = \frac{H}{P}$ و $P_2 = \frac{W}{P}$ به ترتیب تعداد پچ‌ها در راستای ارتفاع و عرض تصویر هستند.

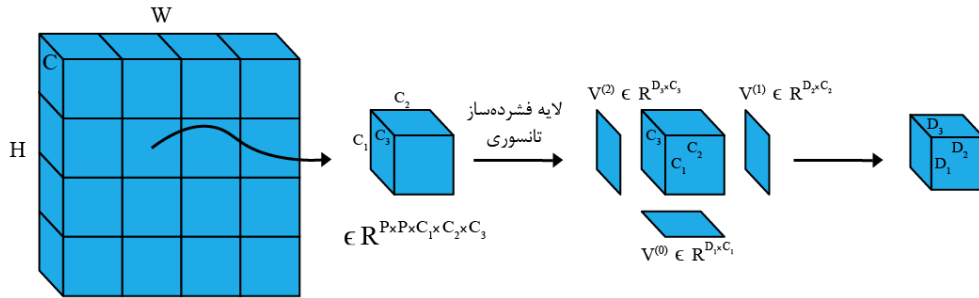
• $C_1 = P$ و $C_2 = P$ ابعاد مکانی هر پچ‌اند.

• $C_3 = C$ تعداد کانال‌های تصویر ورودی است.

در این بازنمایی، هر پچ در موقعیت (i, j) به صورت یک تانسور سه‌بعدی با ابعاد $C_1 \times C_2 \times C_3$ نمایش داده می‌شود. این ساختار چندبعدی امکان استفاده‌ی مستقیم از عملیات تانسوری بدون نیاز به تخت‌سازی را فراهم می‌سازد.

برای نگاشت هر پچ به فضای نهفته، به جای استفاده از MLP از یک لایه‌ی فشرده‌ساز تانسوری استفاده می‌شود. در واقع سه بعد آخر با استفاده از لایه فشرده‌ساز تانسوری تعبیه می‌شود.

^{۲۱}Batch Size
^{۲۲}rearrangement



شکل ۳.۳: تعبیه پیچ‌ها

$$\mathcal{Z} \in \mathbb{R}^{B \times P_1 \times P_2 \times D_1 \times D_2 \times D_3}$$

در این ساختار، هر پیچ به جای تبدیل به یک بردار تخت، به یک تانسور سه‌بعدی در فضای نهفته تبدیل می‌شود. این تانسور ساختار درونی پیچ را حفظ می‌کند.

فرمول‌بندی ریاضی عملیات فشرده‌سازی

فرض می‌کنیم $\mathcal{X}_{patch} \in \mathbb{R}^{C_1 \times C_2 \times C_3}$ نمایانگر یک پیچ ورودی باشد. برای فشرده‌سازی این تانسور به فضای نهفته (D_1, D_2, D_3) ، از سه ماتریس فشرده‌سازی قابل آموزش استفاده می‌شود:

$$V^{(0)} \in \mathbb{R}^{D_1 \times C_1}, \quad V^{(1)} \in \mathbb{R}^{D_2 \times C_2}, \quad V^{(2)} \in \mathbb{R}^{D_3 \times C_3}$$

عملیات فشرده‌سازی با استفاده از ضرب‌های چندحالت^{۲۳} صورت می‌گیرد:

$$\mathcal{X}_{emb} = \mathcal{X}_{patch} \times_0 V^{(0)} \times_1 V^{(1)} \times_2 V^{(2)} \quad (9.3)$$

که در آن $\mathcal{X}_{emb} \in \mathbb{R}^{D_1 \times D_2 \times D_3}$ نگاشت نهفته‌ی تانسوری برای آن پیچ خواهد بود.

n-mode product^{۲۳}

در این مدل ساختار چندبعدی به پیچ‌ها داده می‌شود و همچنین با استفاده از تانسور تعداد پارامترها کاهش پیدا می‌کنند و همچنین ظرفیت مدل نیز افزایش پیدا می‌کند.

در نتیجه، خروجی نهایی مرحله‌ی تعبیه پیچ به صورت زیر تعریف می‌شود:

$$\mathcal{Z} \in \mathbb{R}^{B \times P_1 \times P_2 \times D_1 \times D_2 \times D_3}$$

که در آن هر پیچ به صورت یک تانسور کم‌بعد در فضای نهفته نمایش داده شده است.

۷.۰.۳ ماژول خودتوجهی چندسری مبتنی بر پنجره به صورت تانسوری

همان‌طور که در فصل قبل بیان شد در ساختار مبدل پنجره‌ای، جهت کاهش پیچیدگی محاسباتی، از روشی با عنوان ^{۲۴} بهره گرفته می‌شود. در این روش، ویژگی‌های مکانی تصویر به پنجره‌های کوچک با اندازه‌ی ثابت (بدون هم‌پوشانی) تقسیم شده و سپس عملیات خودتوجهی به صورت محلی و درون هر پنجره مستقل انجام می‌گیرد.

در روش تانسوری، به جای تولید بردارهای تخت‌شده‌ی Q ، K و V ، این بردارها به صورت تانسورهای چندبعدی با استفاده از لایه فشرده‌ساز تانسوری تولید می‌شوند.

ساختار ورودی و تقسیم به پنجره‌ها

خروجی مرحله تعبیه به صورت تانسوری به صورت زیر است:

$$\mathcal{X} \in \mathbb{R}^{B \times H \times W \times D_1 \times D_2 \times D_3}$$

در گام اول، همانند مبدل‌های پنجره‌ای به پنجره‌های $w \times w$ تقسیم می‌شوند. و خروجی ما به

صورت زیر خواهد بود:

^{۲۴} Window-based Multi-Head Self-Attention

$$\mathcal{X}_{win} \in \mathbb{R}^{B \times N_H \times N_W \times w \times w \times D_1 \times D_2 \times D_3}$$

که در آن $N_H = \frac{H}{w}$ و $N_W = \frac{W}{w}$ به ترتیب تعداد پنجره‌ها در راستای ارتفاع و عرض تصویر هستند.

۲. محاسبه‌ی K و Q با استفاده از لایه فشرده‌ساز تانسوری

برای هر پنجره‌ی مکانی، سه لایه‌ی فشرده‌سازی تانسوری مستقل (TCL) برای استخراج تانسورهای K و Q طراحی شده‌اند. فرض می‌شود تانسور ورودی هر پنجره دارای ابعاد:

$$\mathcal{X}_{patch} \in \mathbb{R}^{w \times w \times D_1 \times D_2 \times D_3}$$

باشد. با اعمال سه عملیات ضرب مد- n به ترتیب در ابعاد تانسوری، نگاشت‌های K و Q به صورت زیر حاصل می‌شوند:

$$\mathcal{Q} = \mathcal{X}_{patch} \times_2 V_Q^{(1)} \times_3 V_Q^{(2)} \times_4 V_Q^{(3)} \quad (10.3)$$

$$\mathcal{K} = \mathcal{X}_{patch} \times_2 V_K^{(1)} \times_3 V_K^{(2)} \times_4 V_K^{(3)} \quad (11.3)$$

$$\mathcal{V} = \mathcal{X}_{patch} \times_2 V_V^{(1)} \times_3 V_V^{(2)} \times_4 V_V^{(3)} \quad (12.3)$$

که در آن هر $V^{(i)}$ ماتریس فشرده‌سازی با ابعاد $\mathbb{R}^{D_i \times D_i}$ است. بنابراین، خروجی هر لایه به فضای تانسوری مشابه با ورودی بازمی‌گردد:

$$\mathcal{Q}, \mathcal{K}, \mathcal{V} \in \mathbb{R}^{w \times w \times D_1 \times D_2 \times D_3}$$

۳. تقسیم سرهای توجه چندگانه

برای پیاده‌سازی مکانیزم چندسر^{۲۵}، ابعاد تانسوری خروجی به بخش‌های کوچک‌تری تقسیم می‌شود. فرض می‌شود:

$$D_1 = h_1 \cdot d_1, \quad D_2 = h_2 \cdot d_2, \quad D_3 = h_3 \cdot d_3$$

که در آن:

• h_1, h_2, h_3 تعداد سرها در هر بُعد تانسوری،

• d_1, d_2, d_3 ابعاد نهفته‌ی هر سر در هر بُعد هستند.

با استفاده از عملیات بازآرایی، تانسورهای Q و K به شکل زیر سازمان‌دهی می‌شوند:

$$Q_{head} \in \mathbb{R}^{B \times N_H \times N_W \times w \times w \times h_1 \times h_2 \times h_3 \times d_1 \times d_2 \times d_3}$$

و ساختار مشابهی برای K و V در نظر گرفته می‌شود.

۴. محاسبه ماتریس توجه تانسوری

عملیات ضرب داخلی تعمیم‌یافته بین تانسورهای Q و K برای محاسبه‌ی ماتریس توجه انجام می‌شود. برای هر سر (a, b, c) و برای موقعیت‌های (i, j) و (k, l) در پنجره، مقدار توجه به‌صورت زیر محاسبه می‌شود:

$$\text{Attention}_{ijkl}^{abc} = \sum_{x=1}^{d_1} \sum_{y=1}^{d_2} \sum_{z=1}^{d_3} Q_{ij}^{abcxyz} \cdot K_{kl}^{abcxyz} \quad (۱۳.۳)$$

که نتیجه نهایی یک ماتریس توجه با ابعاد زیر است:

$$\text{Attention} \in \mathbb{R}^{B \times N_H \times N_W \times h_1 \times h_2 \times h_3 \times w^2 \times w^2}$$

برای پایدارسازی محاسبات، نرمال‌سازی زیر اعمال می‌شود:

$$\text{scale} = \frac{1}{\sqrt{d_1 d_2 d_3}} \quad (۱۴.۳)$$

و از نسخه علامت‌دار تابع softmax برای اعمال توجه استفاده می‌گردد:

$$\text{Attention}_{\text{soft}} = \text{sign}(A) \cdot \text{softmax}(|A|) \quad (۱۵.۳)$$

۵. اعمال توجه و بازسازی ویژگی

پس از محاسبه ماتریس توجه، تانسور خروجی هر پنجره با استفاده از ترکیب توجه و تانسور V محاسبه می‌شود:

$$\mathcal{Y}_{ij}^{abcxyz} = \sum_{k,l} \text{Attention}_{ijkl}^{abc} \cdot V_{kl}^{abcxyz} \quad (۱۶.۳)$$

و پس از انجام توجه برای هر پنجره دوباره بازآرایی می‌شوند و به حالت اولیه بازمی‌گردند و ابعاد آن به صورت زیر می‌شود:

$$\mathcal{Y} \in \mathbb{R}^{B \times H \times W \times D_1 \times D_2 \times D_3}$$

در این مدل چون سرها در سه بعد تقسیم می‌شوند در نتیجه این مدل آزادی خیلی بیشتری دارد.

۸.۰.۳ توجه علامت‌دار

در مکانیزم‌های استاندارد خودتوجهی که در مبدل‌ها مورد استفاده قرار می‌گیرند، ماتریس‌های Q ، K و V با یکدیگر تعامل دارند تا وزن‌های توجه استخراج شوند. در این فرآیند، بردارهای Q ^{۲۶} و K ^{۲۷} با استفاده از ضرب داخلی محاسبه می‌شوند و نتیجه برای هر توکن، بیان‌گر میزان اهمیت نسبی سایر توکن‌ها در ارتباط با آن توکن است.

فرمول رایج محاسبه توجه به‌صورت زیر تعریف می‌شود:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V \quad (۱۷.۳)$$

در این ساختار، خروجی QK^T می‌تواند شامل مقادیر مثبت، منفی یا صفر باشد؛ اما پس از اعمال تابع Softmax، تمامی مقادیر به‌صورت نرمال‌سازی‌شده و مثبت خواهند بود. در نتیجه، اطلاعات قطبیت (علامت) که در نتیجهی ضرب داخلی بین بردارهای Q و K وجود داشت، از بین می‌رود. این ویژگی باعث محدود شدن ظرفیت مدل در شناسایی «رابطه منفی یا متضاد» میان توکن‌ها می‌شود.

برای حفظ این اطلاعات قطبیت، از رویکردی با عنوان توجه علامت‌دار استفاده شده است. ایده‌ی اصلی این روش آن است که پس از محاسبه‌ی امتیازهای توجه $A = QK^T$ ، عملیات نرمال‌سازی Softmax بر قدرمطلق این امتیازها انجام شده و سپس علامت اولیه آن‌ها به نتیجه بازگردانده می‌شود.

فرمول این روش به‌صورت زیر تعریف می‌شود:

$$\text{Attention}_{\text{sign}} = \text{sign}(A) \cdot \text{Softmax}(|A|) \quad (۱۸.۳)$$

Query^{۲۶}
Key^{۲۷}

که در آن:

• $A = QK^T$ ماتریس امتیازهای اولیه توجه است.

• $\text{sign}(A)$ عملیاتی است که علامت هر مقدار را حفظ می‌کند (مقادیر $+1$ ، -1 یا 0).

• $|A|$ قدرمطلق مقادیر است که شدت شباهت را بدون در نظر گرفتن جهت نشان می‌دهد.

برای درک بهتر تفاوت میان مکانیزم‌های توجه معمولی و توجه علامت‌دار یک مثال عددی ساده اما گویای زیر ارائه می‌شود.

فرض کنیم بردار پرس‌وجو (Q) و دو بردار کلید (K) به‌صورت زیر تعریف شده‌اند:

$$Q = [1, 2, -1], \quad K_1 = [1, 0, -1], \quad K_2 = [-1, 1, 1]$$

محاسبه امتیاز توجه ابتدا شباهت میان Q و هر کلید با استفاده از ضرب داخلی محاسبه می‌شود:

$$Q \cdot K_1 = (1)(1) + (2)(0) + (-1)(-1) = 1 + 0 + 1 = 2$$

$$Q \cdot K_2 = (1)(-1) + (2)(1) + (-1)(1) = -1 + 2 - 1 = 0$$

بنابراین بردار امتیاز توجه به‌صورت زیر حاصل می‌شود:

$$A = [2, 0]$$

الف) توجه معمولی: در توجه معمولی، از تابع softmax مستقیماً روی مقادیر A استفاده می‌شود:

$$\text{Softmax}(A) = \left[\frac{e^2}{e^2 + e^0}, \frac{e^0}{e^2 + e^0} \right] \approx [0.88, 0.12]$$

در اینجا، هر دو کلید مقدار وزن مثبت می‌گیرند، حتی اگر امتیاز شباهت دومی برابر با صفر بوده باشد. مدل نمی‌تواند تفاوت میان بی‌اثر بودن و تأثیر منفی یا متضاد را به‌خوبی درک کند.

ب) توجه علامت‌دار: در این رویکرد، ابتدا بردار A به دو بخش علامت و قدرمطلق تفکیک می‌شود:

$$\text{sign}(A) = [1, 0], \quad |A| = [2, 0]$$

سپس تابع softmax فقط روی قدرمطلق‌ها اعمال شده و خروجی زیر حاصل می‌شود:

$$\text{Softmax}(|A|) = \left[\frac{e^2}{e^2 + e^0}, \frac{e^0}{e^2 + e^0} \right] \approx [0.88, 0.12]$$

در نهایت، با ضرب عنصر به عنصر با علامت‌ها:

$$\text{Attention}_{\text{sign}} = \text{sign}(A) \cdot \text{Softmax}(|A|) = [0.88, 0.00]$$

در این حالت، کلید دوم به‌طور کامل کنار گذاشته شده است، چرا که امتیاز آن صفر بوده و نشانی از تأثیر مثبت یا منفی ندارد.

در توجه معمولی، تمامی مقادیر صفر یا منفی به وزن‌های مثبت نرمال‌سازی می‌شوند. در مقابل، توجه علامت‌دار توانایی تمایز بین ارتباط مثبت، منفی و خنثی را دارد. این قابلیت می‌تواند در مدل‌سازی دقیق‌تر تضادهای معنایی یا شباهت‌های منفی نقش مهمی ایفا کند.

۹.۰.۳ توجه مبتنی بر پنجره‌های جابه‌جاشده به صورت تانسوری

توجه پنجره‌ای دارای یک محدودیت مهم است: پیچ‌های واقع در پنجره‌های متفاوت هیچ‌گونه تبادل اطلاعاتی ندارند. در حالی که ممکن هست این پیچ‌ها به هم نزدیک باشند به‌منظور رفع این محدودیت، در مبدل‌های پنجره‌ای مکانیزم توجه مبتنی بر پنجره جابه‌جاشده معرفی شده است که در

آن پنجره‌ها در برخی لایه‌ها به صورت جابه‌جاشده تعریف می‌شوند و از ماسک توجه برای جلوگیری از تداخل غیرمجاز استفاده می‌شود.

مراحل دقیق مکانیزم

۱. شیف‌ت چرخشی تانسور ورودی: خروجی مرحله قبلی با یک شیف‌ت چرخشی^{۲۸} به اندازه $\lfloor \frac{w}{2} \rfloor$ پیکسل در راستای افقی و عمودی جابه‌جا می‌شود. این جابه‌جایی باعث می‌شود که پچ‌هایی که پیش‌تر در پنجره‌های جداگانه بودند، اکنون در یک پنجره جدید قرار گیرند. این شیف‌ت روی بعد اول و دوم تانسور که مکان پچ را مشخص می‌کرد صورت می‌گیرد.
۲. اعمال پنجره‌بندی جدید: تانسور جابه‌جاشده به پنجره‌های بدون هم‌پوشانی جدید با اندازه $w \times w$ تقسیم می‌شود. پنجره‌های جدید از موقعیت‌های مختلف تصویر تشکیل شده‌اند.
۳. اعمال خودتوجهی به صورت محلی با ماسک: از آنجا که پنجره‌ها پس از شیف‌ت ممکن است شامل توکن‌هایی از مکان‌های نامرتب باشند، نیاز است که توجه به صورت صحیح انجام شود. یک ماسک دودویی روی ماتریس توجه اعمال می‌شود که فقط اجازه توجه به توکن‌هایی را می‌دهد که واقعاً در یک پنجره معتبر قرار دارند.
- خودتوجهی پنجره جابه‌جاشده باعث ارتباط میان پچ‌های نزدیکی می‌شود که در خودتوجهی پنجره‌ای در یک پنجره قرار نداشته‌اند.

۱۰.۰.۳ ادغام پچ‌ها به صورت تانسوری

در مدل‌های سلسله‌مراتبی بینایی مانند پس از هر مرحله از استخراج ویژگی، لازم است ابعاد مکانی تصویر کاهش یابد تا ویژگی‌ها در سطوح بالاتر به صورت فشرده‌تر و معنایی‌تر نمایش داده شوند. این فرایند با عنوان ادغام پچ‌ها^{۲۹} شناخته می‌شود. در نسخه‌ی اصلی این مدل، ادغام پچ‌ها با استفاده

^{۲۸}cyclic shift

^{۲۹}Patch Merging

از ترکیب ۴ پیچ مجاور (به صورت یک پنجره‌ی 2×2) و سپس اعمال یک لایه‌ی خطی پس از مسطح‌سازی انجام می‌شود. این کار باعث نصف شدن ابعاد مکانی و افزایش ابعاد ویژگی می‌شود. در این پژوهش، رویکرد متفاوتی برای ادغام پیچ‌ها ارائه شده است که مبتنی بر نگاشت تانسوری است. به جای انجام مسطح‌سازی^{۳۰} بر روی پیچ‌ها، از ساختار چندبعدی پیچ‌ها استفاده شده و با کمک یک لایه‌ی فشرده‌سازی تانسوری، ادغام پیچ‌ها بدون از بین رفتن ساختار چندخطی آن‌ها انجام می‌شود.

ساختار ورودی

مانند خروجی مرحله قبلی شبکه دارای ساختار تانسوری زیر باشد:

$$\mathcal{X} \in \mathbb{R}^{B \times H \times W \times R_1 \times R_2 \times C}$$

که در آن:

- B اندازه دسته آموزشی،
- $H \times W$ ابعاد مکانی پیچ‌ها،
- (R_1, R_2, C) ابعاد فضای نهفته‌ی هر پیچ به صورت تانسوری.

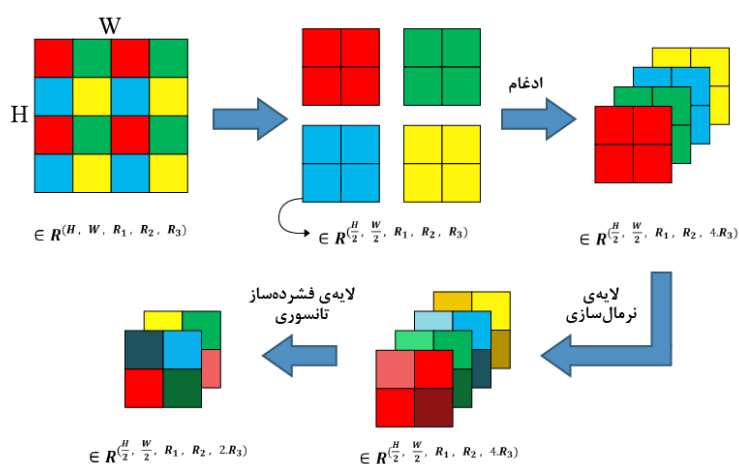
هدف این مرحله، کاهش ابعاد مکانی به $(\frac{H}{2}, \frac{W}{2})$ و در عین حال، افزایش غنای بازنمایی ویژگی‌ها است.

ادغام پیچ‌های مجاور

برای کاهش ابعاد مکانی، ابتدا هر چهار پیچ مجاور در پنجره‌ی 2×2 انتخاب می‌شوند. این چهار پیچ به صورت زیر دسته‌بندی می‌شوند:

^{۳۰} Flatten

- پیچ بالا-چپ، س
- پیچ بالا-راست،
- پیچ پایین-چپ،
- پیچ پایین-راست.



شکل ۴.۳: ادغام پیچ‌ها

در مرحله بعد، این چهار پیچ با یکدیگر ترکیب می‌شوند، به این صورت که هرکدام به صورت مستقل نگه داشته شده و در امتداد بعد کانالی (C) به یکدیگر متصل می‌شوند. در نتیجه، در هر موقعیت مکانی جدید، تانسوری با ابعاد $(R_1, R_2, 4C)$ خواهیم داشت. در سطح کلان، خروجی میان‌مرحله‌ای به صورت زیر خواهد بود:

$$\mathcal{X}_{\text{merged}} \in \mathbb{R}^{B \times \frac{H}{2} \times \frac{W}{2} \times R_1 \times R_2 \times 4C}$$

فشرده‌سازی ویژگی‌های ادغام‌شده

پس از ادغام کانالی، لازم است ابعاد ویژگی‌ها به صورت کنترل‌شده کاهش یابد تا بتوان آن را به شکل ورودی ماژول‌های بعدی تطبیق داد. برای این منظور، از یک لایه‌ی فشرده‌سازی تانسوری استفاده می‌شود که به صورت مستقیم بر روی ابعاد نهفته $(R_1, R_2, 4C)$ عمل می‌کند. برخلاف لایه‌های خطی کلاسیک، در اینجا عملیات فشرده‌سازی در چندین بعد به صورت همزمان و با حفظ ساختار چندخطی انجام می‌شود.

خروجی حاصل، تانسوری با ساختار:

$$\mathcal{Y} \in \mathbb{R}^{B \times \frac{H}{2} \times \frac{W}{2} \times R'_1 \times R'_2 \times C'}$$

که در آن (R'_1, R'_2, C') ابعاد جدید ویژگی‌ها هستند و با توجه به محدودیت طراحی، باید رابطه زیر میان ابعاد قدیم و جدید برقرار باشد:

$$2 \cdot R'_1 \cdot R'_2 \cdot C' = 4 \cdot R_1 \cdot R_2 \cdot C \quad (19.3)$$

این رابطه تضمین می‌کند که ادغام از نظر اطلاعاتی قابل پشتیبانی و از نظر مدل‌سازی متوازن باقی می‌ماند.

۱۱.۰.۳ مرحله‌ی طبقه‌بندی نهایی با استفاده از رگرسیون تانسوری

پس از عبور تصویر ورودی از مراحل سلسله‌مراتبی مختلف شامل تعبیه پیچ، ماژول‌های خودتوجهی تانسوری، و لایه‌های ادغام و فشرده‌سازی، خروجی نهایی آخرین مرحله به صورت یک تانسور چندبعدی در فضای نهفته حاصل می‌شود. این تانسور حاوی بازنمایی غنی از ویژگی‌های سطح بالا و ساختار درونی تصویر است که برای تصمیم‌گیری نهایی مورد استفاده قرار می‌گیرد.

میانگین‌گیری تانسوری در ابعاد مکانی

فرض می‌کنیم خروجی مرحله‌ی نهایی دارای ساختار تانسوری زیر باشد:

$$\mathcal{X} \in \mathbb{R}^{B \times H \times W \times D_1 \times D_2 \times D_3}$$

برای استخراج یک بازنمایی کلی از کل تصویر، میانگین‌گیری در دو بُعد مکانی (H, W) انجام می‌شود. در واقع همانند مبدل پنجره‌ای متحرک از مکان‌های پچ‌ها میانگین‌گیری گرفته می‌شود. این عملیات باعث فشرده‌سازی اطلاعات مکانی و حفظ ساختار تانسوری در فضای ویژگی می‌شود.

$$\bar{\mathcal{X}} = \frac{1}{H \cdot W} \sum_{i=1}^H \sum_{j=1}^W \mathcal{X}_{[:,i,j,:,:,,:]} \in \mathbb{R}^{B \times D_1 \times D_2 \times D_3} \quad (20.3)$$

این عملیات نوعی تجمیع جهانی در ابعاد فضایی محسوب می‌شود که مشابه با میانگین‌گیری سراسری^{۳۱} در شبکه‌های پیچشی عمل می‌کند، با این تفاوت که ساختار تانسوری ویژگی‌ها را حفظ می‌نماید.

۱۲.۰.۳ طبقه‌بندی با استفاده از لایه رگرسیون تانسوری

برای نگاشت خروجی $\bar{\mathcal{X}}$ به برچسب نهایی، از یک لایه رگرسیون تانسوری (TRL) استفاده شده است. این لایه با بهره‌گیری از ضرب داخلی تعمیم‌یافته میان تانسور ورودی و تانسور وزن، نگاشتی خطی از فضای تانسوری به فضای خروجی اعمال می‌کند:

$$\mathbf{Y} = \langle \bar{\mathcal{X}}, \mathcal{W} \rangle_3 + \mathbf{b} \quad (21.3)$$

که در آن:

Global Average Pooling^{۳۱}

- $\bar{\mathcal{X}} \in \mathbb{R}^{B \times D_1 \times D_2 \times D_3}$ بازنمایی فشرده‌ی ورودی،
- $\mathcal{W} \in \mathbb{R}^{D_1 \times D_2 \times D_3 \times C}$ تانسور وزن‌های رگرسیون،
- $\langle \cdot, \cdot \rangle_3$ ضرب داخلی تعمیم‌یافته در سه بعد آخر $\bar{\mathcal{X}}$ و سه بعد اول $\mathcal{W}$ است،
- $\mathbf{b} \in \mathbb{R}^C$ بردار بایاس،
- C تعداد کلاس‌های خروجی است.

فصل ۴

نتایج و تحلیل‌ها

۱.۴ ارزیابی بر روی مجموعه داده CIFAR-10

به منظور ارزیابی عملکرد مدل پیشنهادی، دو پیکربندی مجزا مورد مقایسه قرار گرفتند. در پیکربندی نخست از مدل مبنا یعنی Tiny Swin Transformer به عنوان معماری اصلی بدون تغییرات استفاده شده است. در پیکربندی دوم، نسخه‌ی پیشنهادی یعنی Tensorized Swin Transformer به کار گرفته شده که در آن لایه‌های Patch Embedding، Patch Merging و W-MSA با بهره‌گیری از تکنیک فشرده‌سازی تانسوری بازطراحی شده‌اند.

۱.۱.۴ خلاصه نتایج کمی

جدول ۱.۴ نتایج کمی این دو مدل را در شاخص‌های دقت Top-1 و Top-5 برای داده‌های آموزش و آزمون نشان می‌دهد. در این جدول ترتیب ستون‌ها به گونه‌ای تنظیم شده است که مقایسه میان عملکرد آموزشی و آزمونی به صورت هم‌زمان قابل مشاهده باشد.

جدول ۱.۴: مقایسه عملکرد مدل اصلی و مدل پیشنهادی بر روی مجموعه داده CIFAR-10 بر حسب دقت‌های Top-1 و Top-5.

مدل	# پارامترها	داده آموزش		داده آزمون	
		Top-۱	Top-۵	Top-۱	Top-۵
Tiny Swin	27,528,690	97.48%	99.97%	80.92%	96.45%
Tensorized Swin	1,368,626	80.30%	98.97%	81.80%	99.21%

۲.۱.۴ تحلیل نتایج

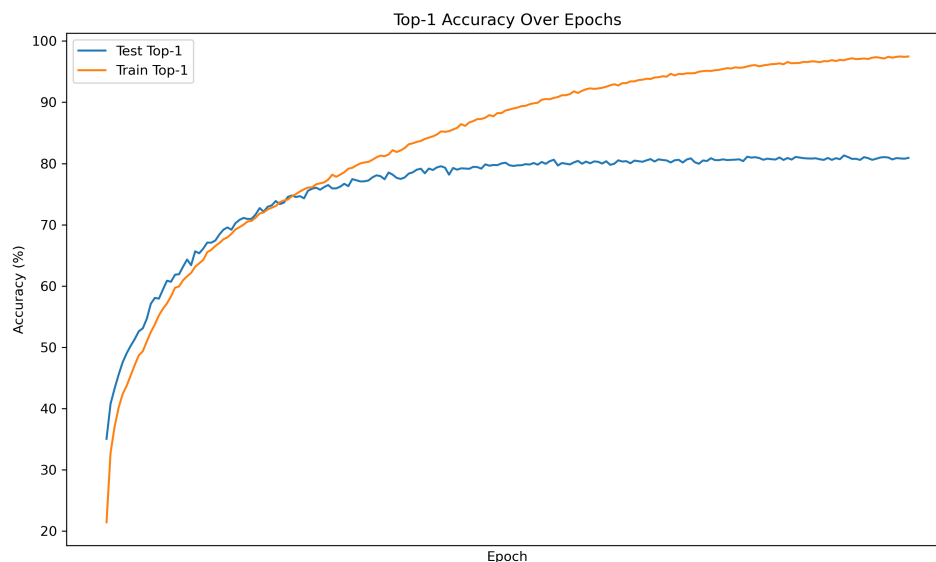
مطابق جدول ۱.۴، مدل پیشنهادی با وجود کاهش چشم‌گیر تعداد پارامترها (از حدود 27.5M به حدود 1.37M، معادل کاهش 95%)، توانسته است دقت آزمون را حفظ یا حتی اندکی بهبود بخشد. در شاخص Top-1، دقت مدل اصلی برابر 80.92% بوده است در حالی که مدل تانسوری به 81.80% رسیده است. همچنین در شاخص Top-5 نیز بهبود محسوسی مشاهده می‌شود، به گونه‌ای که دقت از 96.45% به 99.21% افزایش یافته است.

از نظر تعمیم‌پذیری نیز وضعیت مدل‌ها متفاوت است. مدل اصلی با اختلاف 16.56 واحد درصد میان دقت آموزشی (97.48%) و آزمونی (80.92%) دچار بیش‌برازش بوده است. در مقابل، مدل تانسوری نه تنها دچار این مشکل نشده بلکه دقت آزمون آن اندکی بالاتر از دقت آموزش قرار گرفته است (اختلاف 1.5- واحد درصد). این نتیجه بیانگر وجود نوعی منظم‌سازی ذاتی ناشی از فشرده‌سازی تانسوری و اعمال قیود کم‌مرتبه است. همچنین، بهبود قابل توجه در شاخص Top-5 نشان می‌دهد که مدل پیشنهادی فضای ویژگی غنی‌تری ایجاد کرده است و توانایی پوشش کلاس‌های صحیح در میان پنج پیش‌بینی برتر را دارد.

۳.۱.۴ نمایش روند آموزش

برای درک بهتر فرایند همگرایی، روند تغییرات دقت Top-1 در طول آموزش برای هر دو مدل در شکل‌های ۱.۴ و ۲.۴ نمایش داده شده است. این نمودارها نشان می‌دهند که مدل اصلی به سرعت

به دقت بالایی در داده آموزشی می‌رسد ولی در داده آزمون افت می‌کند، در حالی که مدل تانسوری با روندی یکنواخت‌تر و پایدارتر به دقت نهایی دست پیدا می‌کند.



شکل ۱.۴: روند تغییرات دقت Top-1 مدل اصلی Swin-Tiny بر روی مجموعه داده CIFAR-10.

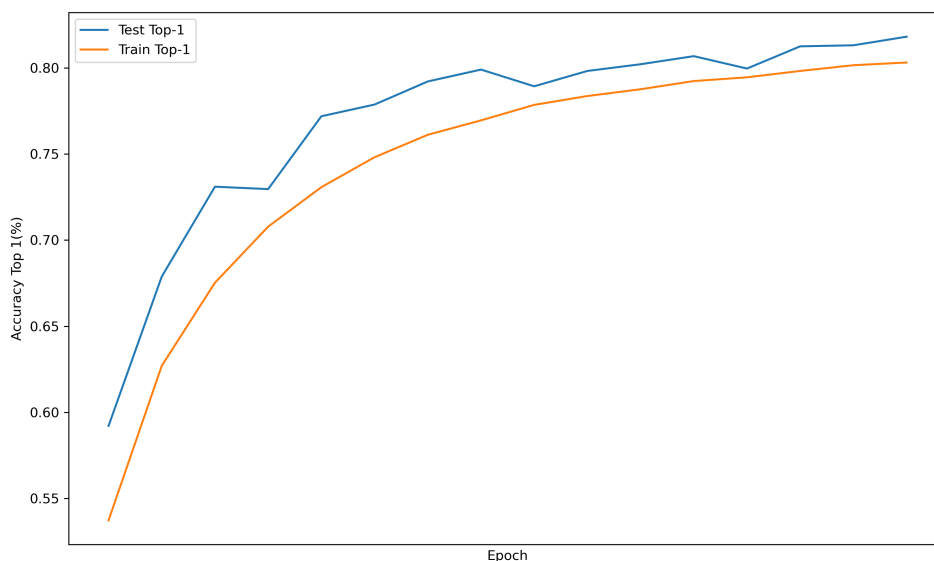
۴.۱.۴ جمع‌بندی

در مجموع می‌توان گفت که استفاده از ساختار تانسوری در بخش‌های کلیدی مدل، علاوه بر کاهش چشم‌گیر پیچیدگی محاسباتی و نیاز حافظه، موجب بهبود تعمیم‌پذیری و کاهش بیش‌برازش نیز شده است. این امر اهمیت به‌کارگیری روش‌های فشرده‌سازی ساختاریافته را در طراحی مدل‌های کارا برای کاربردهای کم‌منبع تأیید می‌کند.

۲.۴ نتایج بر روی دیتاست MNIST

برای ارزیابی عملکرد مدل پیشنهادی بر روی داده‌های ساده‌تر و با ابعاد کوچک‌تر، آزمایش‌ها بر روی دیتاست MNIST انجام شده است. در این بخش نیز دو پیکربندی مقایسه شده‌اند: مدل اصلی

Tiny Swin Transformer و نسخه‌ی پیشنهادی یعنی Tensorized Swin Transformer.



شکل ۲.۴: روند تغییرات دقت Top-1 مدل پیشنهادی Tensorized Swin بر روی مجموعه داده CIFAR-10.

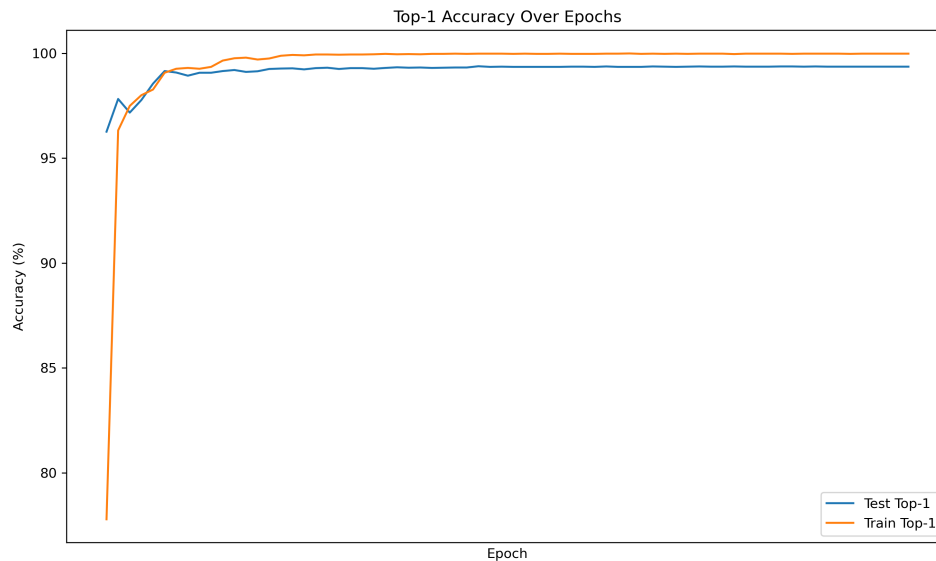
۱.۲.۴ خلاصه نتایج کمی

جدول ۲.۴: مقایسه‌ی عملکرد مدل اصلی و مدل تانسوری بر روی MNIST (فقط ۱-Top و ۵-Top).

مدل	#پارامترها	آموزش		تست	
		Top-۱	Top-۵	Top-۱	Top-۵
Tiny Swin	27,528,690	95.8%	99.9%	97.0%	99.9%
Tensorized Swin	1,368,626	97.3%	99.9%	98.9%	100%

۲.۲.۴ نمایش روند آموزش

شکل‌های ۳.۴ و ۴.۴ روند تغییرات دقت Top-1 را برای مدل اصلی و مدل تانسوری بر روی مجموعه داده‌ی MNIST نشان می‌دهند. هر دو مدل به سرعت به دقت بسیار بالا رسیده‌اند، اما مدل تانسوری با وجود تعداد پارامترهای بسیار کمتر، دقت نهایی بالاتری در داده‌های آزمون کسب کرده است.



شکل ۳.۴: روند دقت Top-1 مدل اصلی Swin-Tiny بر روی MNIST.

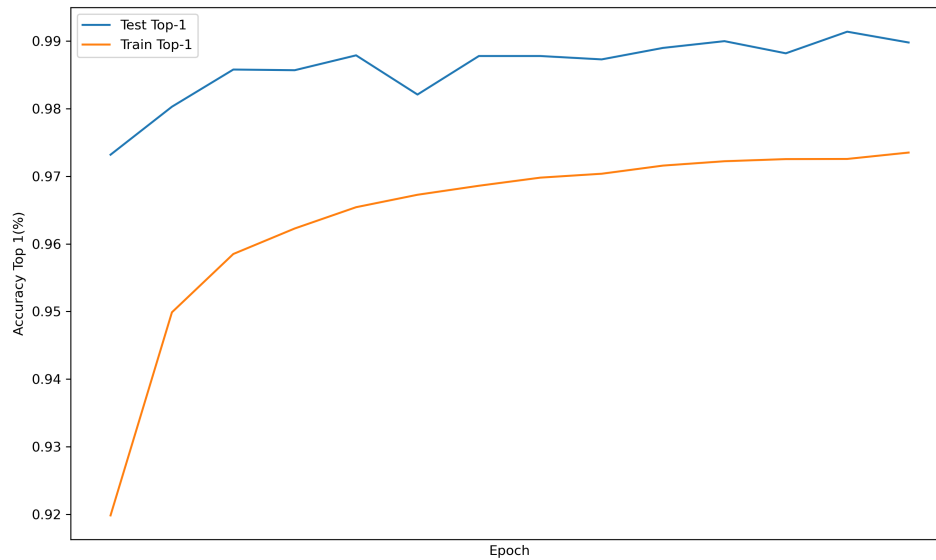
۳.۲.۴ تحلیل و بحث

نتایج جدول ۲.۴ نشان می‌دهد که مدل تانسوری تنها با 1,368,626 پارامتر توانسته است عملکردی بهتر از مدل اصلی با بیش از 27M پارامتر ارائه دهد. دقت آزمون در شاخص Top-1 از 97.0% به 98.9% افزایش یافته و در شاخص Top-5 نیز از 99.9% به 100% رسیده است.

علاوه بر این، شکاف میان دقت آموزش و آزمون در مدل اصلی حدود 1.2 واحد درصد بوده است، در حالی که در مدل تانسوری این شکاف منفی شده و دقت آزمون بالاتر از دقت آموزش قرار گرفته است. این نشان‌دهنده تعمیم‌پذیری بهتر مدل تانسوری است. حتی در مجموعه داده‌ای ساده مانند MNIST نیز این بهبود اهمیت دارد، زیرا تأکید می‌کند که فشرده‌سازی تانسوری نه تنها پارامترها را کاهش می‌دهد، بلکه می‌تواند به عنوان یک منظم‌ساز مؤثر عمل کند.

۴.۲.۴ جمع‌بندی

به طور کلی، نتایج آزمایش‌ها بر روی دیتاست MNIST نشان دادند که مدل تانسوری با کاهش چشمگیر پارامترها نه تنها عملکرد مدل را حفظ کرده بلکه در داده‌های آزمون نیز عملکرد بهتری



شکل ۴.۴: روند دقت Top-1 مدل تانسوری پیشنهادی بر روی MNIST.

داشته است. این نتایج اهمیت استفاده از روش‌های تانسوری را حتی در داده‌های ساده تأیید می‌کند.

۳.۴ نتایج بر روی دیتاست Tiny ImageNet

برای بررسی عملکرد مدل در یک سناریوی چالش‌برانگیزتر، آزمایش‌هایی بر روی دیتاست Tiny ImageNet انجام شد. در این بخش علاوه بر مدل اصلی Tiny Swin Transformer، مدل پیشنهادی تانسوری نیز با دو بهینه‌ساز متفاوت (Adam و AdamW) مورد ارزیابی قرار گرفته است.

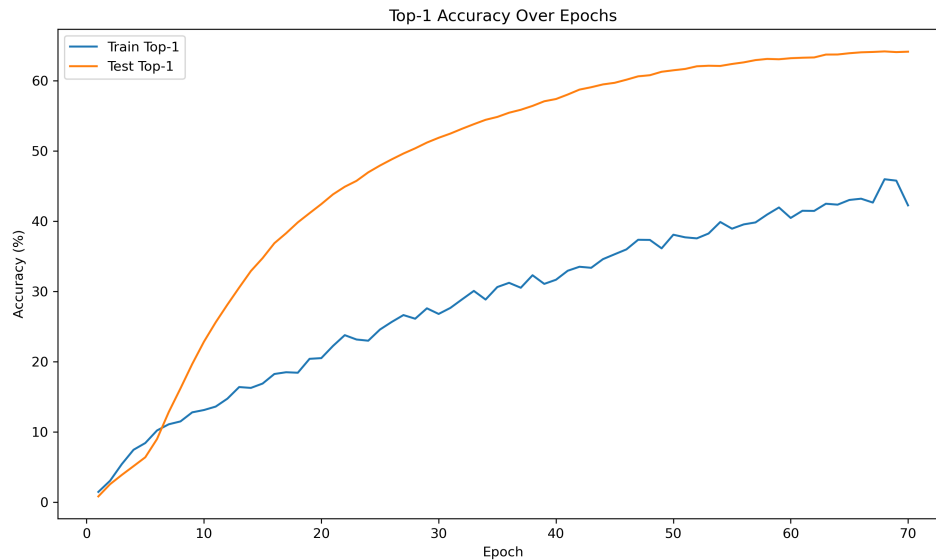
۱.۳.۴ خلاصه نتایج کمی

جدول ۳.۴: مقایسه‌ی عملکرد مدل‌ها بر روی Tiny ImageNet.

بهینه‌ساز	مدل	# پارامتر	آموزش			
			Top-۱	Top-۵	Top-۱	Top-۵
—	Tiny Swin	27,528,690	42.26%	59.03%	64.14%	85.42%
Adam	Tensorized Swin	1,368,626	83.89%	98.50%	30.85%	54.17%
AdamW	Tensorized Swin	1,368,626	55.30%	80.90%	35.90%	62.07%

۲.۳.۴ نمایش روند آموزش

شکل ۵.۴ روند تغییر دقت Top-1 را برای مدل اصلی Tiny Swin نشان می‌دهد. همچنین، شکل‌های ۶.۴ و ۷.۴ مربوط به مدل تانسوری با بهینه‌سازهای Adam و AdamW هستند.

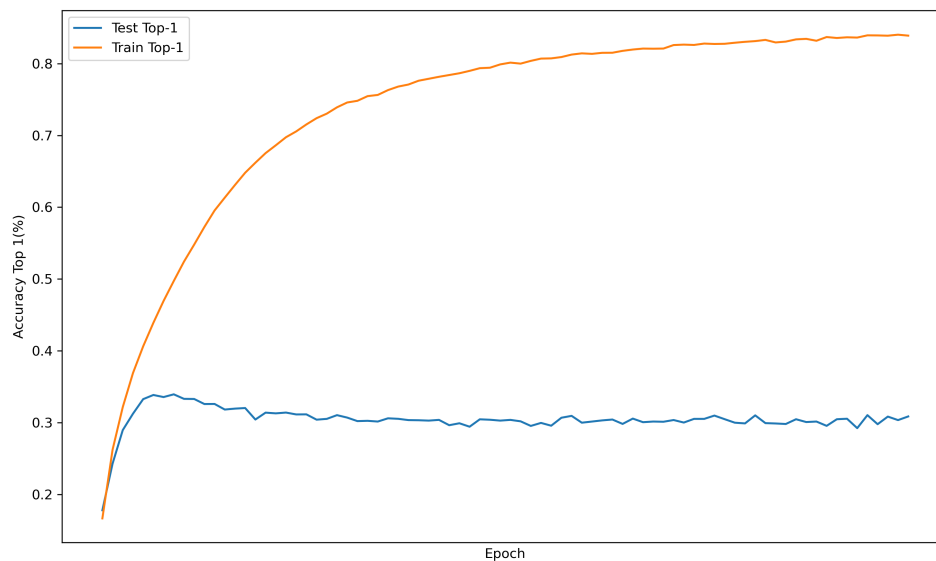


شکل ۵.۴: روند دقت Top-1 مدل اصلی Swin-Tiny بر روی Tiny ImageNet.

۳.۳.۴ تحلیل و بحث

نتایج جدول ۳.۴ و نمودارهای مربوطه نشان می‌دهد که رفتار مدل‌ها در این دیتاست متفاوت از دو دیتاست قبلی است. مدل اصلی Tiny Swin با داشتن بیش از 27M پارامتر، دقت آزمون Top-1 معقولی معادل 64.14% به دست آورده است. با این حال اختلاف میان دقت آموزش (42.26%) و آزمون بسیار زیاد است که می‌تواند ناشی از محدودیت در داده‌ها یا تنظیمات بهینه‌سازی باشد.

مدل تانسوری با بهینه‌ساز Adam عملکردی متفاوت نشان داده است. این مدل در داده‌های آموزش به دقت بسیار بالایی (83.89%) رسیده اما در داده‌های آزمون تنها 30.85% کسب کرده است، که نشانگر بیش‌برازش شدید است. در مقابل، استفاده از بهینه‌ساز AdamW منجر به بهبود نسبی عملکرد شده و دقت آزمون به 35.90% افزایش یافته است، هرچند همچنان فاصله‌ی زیادی تا



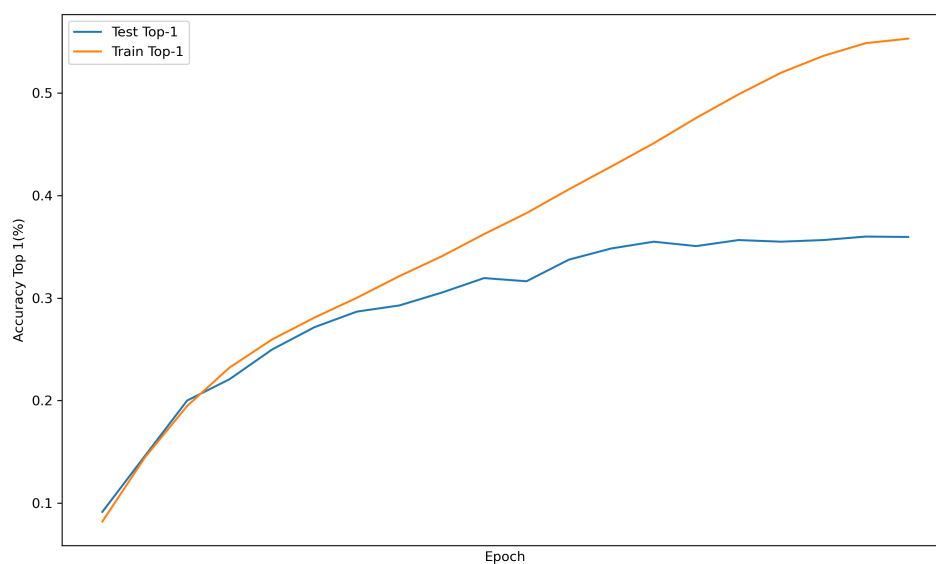
شکل ۴.۶: روند دقت Top-1 مدل تانسوری با بهینه‌ساز Adam بر روی Tiny ImageNet.

عملکرد مدل اصلی دارد.

این یافته‌ها بیانگر آن است که کاهش شدید پارامترها در دیتاست‌های پیچیده‌تر مانند Tiny ImageNet، در صورت عدم استفاده از منظم‌سازی و استراتژی‌های بهینه‌سازی مناسب، می‌تواند به افت تعمیم‌پذیری منجر شود.

۴.۳.۴ جمع‌بندی

در مجموع، در حالی که نتایج به‌دست‌آمده روی دو دیتاست CIFAR-10 و MNIST نشان دادند که مدل تانسوری می‌تواند همزمان با کاهش چشمگیر پارامترها عملکرد بهتری نیز ارائه دهد، در دیتاست چالش‌برانگیزتر Tiny ImageNet این کاهش ظرفیت موجب افت دقت آزمون شده است. با این حال استفاده از بهینه‌ساز مناسب مانند AdamW توانسته است تا حدی شکاف میان آموزش و آزمون را کاهش دهد و مسیری برای تحقیقات آینده در ترکیب روش‌های فشرده‌سازی و بهینه‌سازی پیشنهاد می‌کند.



شکل ۷.۴: روند دقت Top-1 مدل تانسوری با بهینه‌ساز AdamW بر روی Tiny ImageNet.

کتاب‌نامه

- [1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. [28](#), [29](#)
- [2] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. Neural machine translation by jointly learning to align and translate. In *Proceedings of the 2015 International Conference on Learning Representations (ICLR)*, San Diego, CA, 2015. [16](#), [17](#), [30](#), [31](#)
- [3] Yoshua Bengio et al. *Learning long-term dependencies with gradient descent is difficult*. IEEE Transactions on Neural Networks, 1994. [26](#), [27](#)
- [4] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006. [4](#), [5](#), [7](#), [11](#), [12](#), [15](#)
- [5] Peter F Brown, Vincent J. Della Pietra, Stephen A. Della Pietra, and Robert L Mercer. The mathematics of statistical machine translation: Parameter estimation. *Computational linguistics*, 19(2):263–311, 1993. [16](#)
- [6] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20(3):273–297, 1995. [10](#)
- [7] Thomas M. Cover and Peter E. Hart. Nearest neighbor pattern classification. *IEEE Transactions on Information Theory*, 13(1):21–27, 1967. [9](#), [10](#)
- [8] Daniel Crevier. *AI: The Tumultuous History of the Search for Artificial Intelligence*. Basic Books, New York, 1993. [3](#)

- [9] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. [16](#), [37](#), [38](#)
- [10] Pedro Domingos and Michael Pazzani. On the optimality of the simple bayesian classifier under zero-one loss. *Machine Learning*, 29(2–3):103–130, 1997. [11](#), [12](#)
- [11] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, et al. An image is worth 16x16 words: Transformers for image recognition at scale, 2020. [15](#), [33](#), [35](#), [36](#), [37](#), [38](#), [39](#), [40](#)
- [12] Richard O. Duda and Peter E. Hart. *Pattern Classification and Scene Analysis*. John Wiley & Sons, New York, 1973. [9](#), [10](#)
- [13] Jeffrey L. Elman. Finding structure in time. *Cognitive Science*, 14(2):179–211, 1990. [17](#), [21](#)
- [14] Edward A. Feigenbaum and Pamela McCorduck. *The Fifth Generation: Artificial Intelligence and Japan's Computer Challenge to the World*. Addison-Wesley, Reading, MA, 1983. [3](#)
- [15] Felix A. Gers, Jürgen Schmidhuber, and Fred Cummins. Learning to forget: Continual prediction with lstm. In *Proceedings of the Ninth International Conference on Artificial Neural Networks (ICANN-99)*, pages 850–855, Edinburgh, UK, 1999. [15](#)
- [16] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, Cambridge, MA, 2016. [4](#)
- [17] Safa Hamreras, Sukhbinder Singh, and Román Orús. Tensorization as a powerful tool for compression and interpretability in neural networks: A position paper. *arXiv preprint arXiv:2505.20132*, 2025. [58](#)
- [18] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 770–778, 2016. [14](#), [26](#), [27](#), [33](#), [39](#), [40](#), [47](#), [49](#)
- [19] Dan Hendrycks and Kevin Gimpel. Gaussian error linear units (gelus). *arXiv preprint arXiv:1606.08415*, 2016. [47](#)

- [20] Sepp Hochreiter. The vanishing gradient problem during learning recurrent neural nets and problem solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2):107–116, 1998.
- [21] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997. [15](#), [21](#), [26](#), [27](#)
- [22] John Hutchins. *Machine translation: past, present, future*. Ellis Horwood Chichester, 1986. [16](#)
- [23] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International conference on machine learning*, pages 448–456, 2015. [28](#), [29](#)
- [24] Gareth James, Daniela Witten, Trevor Hastie, and Robert Tibshirani. *An Introduction to Statistical Learning: with Applications in R*. Springer, New York, 2013. [5](#), [10](#)
- [25] Shibo Jie and Zhi-Hong Deng. Fact: Factor-tuning for lightweight adaptation on vision transformer. In *Proceedings of the AAAI Conference on Artificial Intelligence*, 2023. 0.01% ViT . [59](#)
- [26] Philipp Koehn. *Statistical Machine Translation*. Cambridge University Press, 2010. [16](#)
- [27] Philipp Koehn, Franz Josef Och, and Daniel Marcu. Statistical phrase-based translation. In *Proc. of NAACL/HLT*, pages 48–54, 2003. [17](#)
- [28] Tamara G Kolda and Brett W Bader. Tensor decompositions and applications. *SIAM review*, 51(3):455–500, 2009. [54](#)
- [29] Jean Kossaifi, Aran Khanna, Zachary C. Lipton, Tommaso Furlanello, and Anima Anandkumar. Tensor contraction layers for parsimonious deep nets. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, 2017. [52](#), [53](#), [55](#), [56](#), [58](#)

- [30] Jean Kossaifi, Zachary C. Lipton, Arinbjörn Kolbeinsson, Aran Khanna, Tommaso Furlanello, and Anima Anandkumar. Tensor regression networks. *Journal of Machine Learning Research*, 21(123):1–21, 2020. [52](#), [53](#), [54](#), [55](#), [56](#), [57](#), [58](#)
- [31] Jean Kossaifi, Yannis Panagakis, Anima Anandkumar, and Maja Pantic. Tensor contraction layers for parsimonious deep nets. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 1940–1949, 2017. [55](#)
- [32] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in neural information processing systems*, pages 1097–1105, 2012. [14](#), [33](#)
- [33] Vadim Lebedev, Yaroslav Ganin, Maksim Rakhuba, Ivan V. Oseledets, and Victor S. Lempitsky. Speeding-up convolutional neural networks using fine-tuned cp-decomposition. In *International Conference on Learning Representations (ICLR)*, 2015. [53](#)
- [34] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998. [13](#), [14](#), [33](#)
- [35] James Lighthill. *Artificial Intelligence: A General Survey*. HM Stationery Office, London, 1973. Science Research Council Report. [3](#)
- [36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proc. of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pages 10012–10022, 2021. [39](#), [41](#), [44](#), [45](#), [46](#), [47](#), [48](#), [49](#), [50](#)
- [37] Minh-Thang Luong, Hieu Pham, and Christopher D Manning. Effective approaches to attention-based neural machine translation. In *Proc. of EMNLP*, pages 1412–1421, 2015. [17](#)
- [38] Andrew McCallum and Kamal Nigam. A comparison of event models for naive bayes text classification. In *AAAI-98 Workshop on Learning for Text Categorization*, pages 41–48, Madison, WI, 1998. [11](#), [12](#)

- [39] John McCarthy, Marvin Minsky, Nathaniel Rochester, and Claude E. Shannon. A proposal for the dartmouth summer research project on artificial intelligence. *Dartmouth College AI Archive*, 1956. [2](#)
- [40] Pamela McCorduck. *Machines Who Think: A Personal Inquiry into the History and Prospects of Artificial Intelligence*. A. K. Peters, Ltd., Natick, MA, 2nd edition, 2004. [3](#)
- [41] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg S Corrado, and Jeffrey Dean. Distributed representations of words and phrases and their compositionality. In *Advances in neural information processing systems*, pages 3111–3119, 2013. [19](#)
- [42] Tom M. Mitchell. *Machine Learning*. McGraw-Hill, New York, 1997. [4](#), [9](#), [10](#), [11](#)
- [43] Douglas C. Montgomery, Elizabeth A. Peck, and Geoffrey G. Vining. *Introduction to Linear Regression Analysis*. John Wiley & Sons, Hoboken, NJ, 6th edition, 2021. [6](#)
- [44] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, Cambridge, MA, 2012. [4](#), [5](#), [10](#), [11](#), [12](#)
- [45] Makoto Nagao. A framework of a mechanical translation between japanese and english by analogy principle. In *Proc. of the international NATO symposium on artificial and human intelligence*, pages 173–180, 1984. [16](#)
- [46] Allen Newell, J. Clifford Shaw, and Herbert A. Simon. Report on a general problem-solving program. In *Proceedings of the International Conference on Information Processing*, pages 256–264, 1959. [3](#)
- [47] Nils J. Nilsson. *The Quest for Artificial Intelligence: A History of Ideas and Achievements*. Cambridge University Press, Cambridge, 2010. [3](#), [4](#)
- [48] Alexander Novikov, Dmitry Podoprikin, Anton Osokin, and Dmitry P. Vetrov. Tensorizing neural networks. In *Advances in Neural Information Processing Systems*, volume 28, pages 442–450, 2015. [51](#), [55](#), [58](#)
- [49] Jeffrey Pennington, Richard Socher, and Christopher D Manning. Glove: Global vectors for word representation. In *Proc. of EMNLP*, pages 1532–1543, 2014. [19](#)

- [50] Alec Radford, Karthik Narasimhan, Tim Salimans, and Ilya Sutskever. Improving language understanding by generative pre-training. OpenAI report, 2018. [16](#)
- [51] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986. [12](#), [15](#)
- [52] Stuart J. Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, London, 3rd edition, 2016. [3](#), [4](#)
- [53] Karen Simonyan and Andrew Zisserman. Very deep convolutional networks for large-scale image recognition. In *International Conference on Learning Representations (ICLR)*, 2015. arXiv:1409.1556. [14](#)
- [54] Ilya Sutskever, Oriol Vinyals, and Quoc V Le. Sequence to sequence learning with neural networks. In *Advances in neural information processing systems*, pages 3104–3112, 2014. [17](#), [30](#), [31](#)
- [55] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, MA, 2nd edition, 2018. [7](#)
- [56] Christian Szegedy, Wei Liu, Yangqing Jia, Pierre Sermanet, Scott Reed, Dragomir Anguelov, Dumitru Erhan, Vincent Vanhoucke, and Andrew Rabinovich. Going deeper with convolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 1–9, 2015. [14](#)
- [57] Ledyard R Tucker. Some mathematical notes on three-mode factor analysis. *Psychometrika*, 31(3):279–311, 1966. [54](#)
- [58] Vladimir Vapnik. *Statistical Learning Theory*. Wiley, New York, 1998. [10](#)
- [59] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. [16](#), [17](#), [20](#), [21](#), [22](#), [26](#), [27](#), [28](#), [29](#), [30](#), [31](#), [33](#), [36](#), [38](#), [39](#)
- [60] Yinchong Yang, Denis Krompass, and Volker Tresp. Tensor-train recurrent neural networks for video classification. In *Proceedings of the 34th International Conference on Machine Learning (ICML)*, pages 3891–3900, 2017. [54](#)

پیوست آ

جزئیات مدل‌ها و جدول پارامترها

Abstract

In recent years, vision transformers have emerged as one of the core architectures in deep learning models for image processing. However, conventional designs of these transformers often rely on fully connected layers that require flattening high-order input tensors into vectors. This process not only leads to a significant increase in the number of parameters but also results in the loss of spatial structures and inter-dimensional relationships within the data. In this study, we propose a tensor-based framework for the design of a window-based vision transformer that leverages Tensor Compression Layers (TCL) and Tensor Regression Layers (TRL) to effectively model multilinear mappings across data dimensions. By preserving the multidimensional structure of images, the proposed method achieves a substantial reduction in the number of parameters while maintaining structural information and improving the interpretability of the model. Experimental results on standard benchmark datasets demonstrate that incorporating tensorized structures not only reduces computational complexity but also enhances classification accuracy.

Key Words: Vision Transformer, Tensor Decomposition, Tensor Compression Layer (TCL), Tensor Regression Layer (TRL), Multilinear Mapping, Image Classification, Deep Learning.



T. M. U.

Vision Trnasformer

A Thesis Presented for the Degree of
Master in Computer Science

Faculty of Mathematical Sciences

Tarbiat Modares University

by

Seyed Mohammad Badzohreh

Supervisor

Dr. Mansoor Rezghi

2024